

这个项目的故事线是这样的：

起因：传统推荐系统用整数 ID 表示商品，存在冷启动、语义缺失、稀疏性三大问题。同时，大语言模型展现出了强大的序列建模和生成能力，但它只能处理离散 Token，没法直接处理推荐系统里的连续向量。

核心矛盾：推荐系统的世界是连续的（向量），大模型的世界是离散的（Token）。怎么把这两个世界连起来？

解决方案：设计一条三步流水线：先用多模态信息为商品生成高质量的连续向量（sasrec），再用RQ-VAE把连续向量翻译成离散的语义 IDToken（rqvae），最后用大模型在语义 ID 序列上做自归生成推荐（gr）。

学术渊源：整个方案参考的是 Google2023 年发表的 TIGER（TrainingItemGeneratorsfor EnhancedRetrieval）架构。

冷启动：新用户、新商品、新内容刚出现时，没有足够历史行为，模型学不到可靠向量，所以推荐效果差。传统 ID 模型如果只依赖商品 ID。

语义缺失：商品 ID 只表示编号，不包含商品标题、图片、类目、风格、用途等信息，模型不能天然理解商品之间的相似关系。

稀疏性：用户和商品数量很大，但真实点击、购买、收藏行为只覆盖了其中很小一部分，大量用户和商品之间没有数据，模型很难学准。

数据集格式：

我们的数据集是用户行为序列格式，每个用户一条记录。每条记录是一个 tuple list，包含 user_id、item_id、用户特征、物品特征、行为类型和时间戳。特征采用稀疏编码，用特征 ID 映射特征值。我们用 indexer 将原始字符串 ID 转为模型可用的整数索引。另外还预提取了多模态 embedding 作为物品的语义特征。工程上我们用 offsets 文件实现大文件的随机读取，避免全量加载内存。"

原始的数据行为日志：

原始日志表									
user_id	item_id	action	timestamp	用户年龄	用户性别	物品类目	物品品牌	物品价格	
u_张三	i_手机A	click	2024-01-01 10:00	25岁	男	手机	华为	3000元	
u_张三	i_耳机B	click	2024-01-01 10:05	25岁	男	耳机	华为	500元	
u_张三	i_手机A	buy	2024-01-01 10:30	25岁	男	手机	华为	3000元	
u_李四	i_衣服C	click	2024-01-01 11:00	30岁	女	上衣	Nike	200元	
u_李四	i_裤子D	click	2024-01-01 11:10	30岁	女	裤子	Nike	300元	

1. 特征编码（将字符串转为数字）

用户特征编码

原始值	编码后的数字ID
年龄 18-24岁 → 1	年龄段编码
年龄 25-30岁 → 2	年龄段编码
年龄 31-40岁 → 3	年龄段编码
性别 男 → 1	性别编码
性别 女 → 2	性别编码

物品特征编码

原始值	编码后的数字ID
类目 手机 → 1	类目编码
类目 耳机 → 2	类目编码
类目 上衣 → 3	类目编码
品牌 华为 → 1	品牌编码
品牌 Nike → 2	品牌编码
价格 0-500元 → 1	价格区间编码
价格 500-2000元 → 2	价格区间编码
价格 2000元以上 → 3	价格区间编码

行为类型编码

原始行为	编码数字
click (点击)	1
favorite (收藏)	2
buy (购买)	3

三、特征字典解释

特征采用 稀疏编码格式：{"特征ID": 特征值}

用户特征

特征ID	含义	示例
103	用户年龄分段	5 → 25-30岁
104	用户性别	2 → 女性
105	用户城市等级	3 → 三线城市
106	用户兴趣标签列表	[1, 5, 8] → 兴趣: 科技/美妆/运动
109	用户会员等级	1 → 普通会员

物品特征

特征ID	含义	示例
100	物品一级类目	10 → 电子产品
117	物品品牌	3 → Apple
111	物品二级类目	5 → 手机
114	物品价格区间	2 → 500-1000元

为什么用数字ID? 数字ID便于编码和索引, 节省存储空间, 特征含义通过 `indexer.pkl` 映射。

这里要用数字进行编码。

为什么这里不用精确地数值, 比如年龄=25 而是年龄 18-24 映射到 1:

推荐里很多时候“区间”比“精确值”更重要, 且为了避免稀疏性, 加快索引速度。特征 id10:3, 本质上是 embedding 的索引位置, 因为是 3 就去向量矩阵中寻找第三行向量来表示该特征。

2. 构建 `indexer.pkl` (ID 映射字典)

```
indexer = {
    # 用户ID映射: 原始ID → 数字索引
    "u": {
        "u_张三": 1,
        "u_李四": 2
    },
    # 物品ID映射: 原始ID → 数字索引
    "i": {
        "i_手机A": 1,
        "i_耳机B": 2,
        "i_衣服C": 3,
        "i_裤子D": 4
    },
    # 特征值映射: 特征ID → [原始值 → 数字索引]
    "f": {
        "103": {"18-24岁": 1, "25-30岁": 2, "31-40岁": 3}, # 年龄特征
        "104": {"男": 1, "女": 2}, # 性别特征
        "100": {"手机": 1, "耳机": 2, "上衣": 3, "裤子": 4}, # 类别特征
        "117": {"华为": 1, "Nike": 2}, # 品牌特征
        "114": {"0-500元": 1, "500-2000元": 2, "2000+": 3} # 价格特征
    }
}
```

3. 构建 seq.jsonl (用户行为序列)

每一行代表 一个用户的完整行为轨迹:

```
["user_001", "item_123", {"103":5,"104":2}, {"100":10,"117":3}, 1, 17000000000],
["user_001", "item_456", {"103":5,"104":2}, {"100":20,"117":5}, 1, 1700000100],
["user_001", "item_789", {"103":5,"104":2}, {"100":15,"117":8}, 1, 170000020]
```

每个元素是一个 Tuple (元组) , 包含6个字段:

字段位置	名称	含义	示例
0	user_id	用户原始ID (字符串)	"user_001"
1	item_id	物品原始ID (字符串)	"item_123"
2	user_feat	用户特征字典	{"103":5, "104":2}
3	item_feat	物品特征字典	{"100":10, "117":3}
4	action_type	行为类型 (点击/购买等)	1 表示点击
5	timestamp	时间戳	17000000000

4. 构建 seq_offsets.pkl（字节偏移）

seq.jsonl 文件内容：

第0行（张三）：字节0 到 字节200

第1行（李四）：字节200 到 字节350

```
seq_offsets = {  
    0: 0, # 用户0（张三）的数据从文件第0字节开始  
    1: 200 # 用户1（李四）的数据从文件第200字节开始  
}
```

作用：想读取李四的数据时，直接跳到第200字节，不用从头扫描文件。

5. 构建 item_feat_dict.json（物品静态特征汇总）

把每个物品的所有特征汇总到一个字典里：

原始物品信息

item_id	类目	品牌	价格	标题	图片
i_手机A	手机	华为	3000元	"华为Mate60旗舰手机"	[图片URL]
i_耳机B	耳机	华为	500元	"华为FreeBuds无线耳机"	[图片URL]
i_衣服C	上衣	Nike	200元	"Nike运动T恤透气款"	[图片URL]
i_裤子D	裤子	Nike	300元	"Nike运动裤宽松版"	[图片URL]

编码后的 item_feat_dict.json

```
{  
    "1": {"100": 1, "117": 1, "114": 3, "111": 1},  
    "2": {"100": 2, "117": 1, "114": 1, "111": 2},  
    "3": {"100": 3, "117": 2, "114": 1, "111": 3},  
    "4": {"100": 4, "117": 2, "114": 2, "111": 4}  
}
```

特征ID含义对照表

特征ID	含义	示例（物品1=手机A）
100	一级类目	1 → 手机
111	二级类目	1 → 旗舰机
117	品牌	1 → 华为
114	价格区间	3 → 2000元以上
118	销量等级	未填充时用默认值0

作用：模型训练时需要知道每个物品的特征。当用户点击了物品1，模型需要查询这个物品的类目、品牌等信息。

6. 构建多模态 Embedding (creative_emb/)

物品有文本标题和图片，需要提取成向量

文本 Embedding 提取

用预训练模型 (如 BERT) 提取标题的语义向量:

物品 i_手机A 的标题: "华为Mate60旗舰手机"
↓ 传入 BERT 模型
↓ 输出 1024 维向量
[0.12, 0.34, -0.56, 0.78, ..., 0.23] (共1024个数字)

图片 Embedding 提取

用视觉模型 (如 ViT/CLIP) 提取图片特征:

物品 i_手机A 的图片: [手机图片]
↓ 传入 ViT 模型
↓ 输出 3584 维向量
[0.08, 0.21, 0.45, -0.33, ..., 0.15] (共3584个数字)

存储格式 (creative_emb/)

```
creative_emb/
├── emb_82_1024/ # 特征ID=82, 维度1024 (文本embedding)
│   ├── part_001.json
│   └── part_002.json
├── emb_83_3584/ # 特征ID=83, 维度3584 (图片embedding)
│   ├── part_001.json
│   └── part_002.json
└── emb_81_32.pkl # 特征ID=81, 维度32 (轻量级embedding)
```

每个 JSON 文件内容:

```
// emb_82_1024/part_001.json
{"anonymous_cid": "i_手机A", "emb": [0.12, 0.34, -0.56, ..., 0.23]}
{"anonymous_cid": "i_耳机B", "emb": [0.08, 0.21, 0.45, ..., 0.15]}
{"anonymous_cid": "i_衣服C", "emb": [0.33, 0.11, -0.22, ..., 0.44]}
```

最终数据汇总：

```
data_dir/
├── seq.jsonl          # 用户行为序列（上面构建的）
├── seq_offsets.pkl    # 字节偏移索引
├── indexer.pkl        # ID映射字典
├── item_feat_dict.json # 物品静态特征汇总
└── creative_emb/      # 多模态embedding（图片/文本提取）
```

"我们的数据集是从用户行为日志构建的。首先把原始的字符串 ID（如用户名、商品名、类目名）编码成数字索引，存到 `indexer.pkl`。然后按用户分组，把每个用户的所有行为按时间排序，生成一条序列记录存到 `seq.jsonl`。每条记录包含：用户索引、物品索引、用户特征、物品特征、行为类型、时间戳。为了支持大规模数据的高效读取，我们还预计算了每个用户记录的文件偏移量存到 `seq_offsets.pkl`。"

除了行为序列，我们还有两类重要数据：

`item_feat_dict.json`：存储每个物品的静态特征（类目、品牌、价格等）。这些是结构化特征，用稀疏编码存储。

`creative_emb/`：存储多模态 embedding。我们用预训练模型（BERT 提取文本、ViT 提取图片）把商品的标题和图片转成向量。比如特征 ID=82 是 1024 维的文本向量，ID=83 是 3584 维的图片向量。

这样模型可以同时利用三种信息：用户行为序列（动态）、物品结构化特征（静态）、多模态语义向量（文本/图片）。这对解决冷启动问题很重要——新物品没有历史交互，但有文本和图片，模型可以通过语义向量理解它。"

物品信息分为了两类存储，一类是静态信息，另一类是图片和名称的向量信息，为什么要分为两类存储而不是合并？

一、数据规模差异巨大

静态特征 (`item_feat_dict.json`)

```
{"i": [{"100":1, "117":1, "114":3}]}
```

 // 每个物品只有几个整数

- 每个物品约 几十字节
- 100万个物品 → 几十MB
- 可以 全部加载到内存

多模态 Embedding (`creative_emb/`)

```
{"i": [0.12, 0.34, ..., 0.23]}
```

 // 每个物品1024-4096个浮点数

- 每个物品约 4KB-16KB (1024维 × 4字节/浮点)
- 100万个物品 → 4GB-16GB
- 不能全部加载，需要 按需读取

二、使用方式不同

类型	使用场景	处理方式
静态特征	传统推荐模型的特征编码	做 embedding lookup，生成稀疏特征向量
多模态 Embedding	语义相似度计算、冷启动	直接拼接或做注意力计算

```
# 静态特征的用法（稀疏特征）
item_feat = {"100": 1, "117": 1} # 类目=1, 品牌=1
category_emb = category_embedding_table[item_feat["100"]] # 查表得:

# 多模态embedding的用法（稠密向量）
text_emb = load_mm_emb("82", "item_1") # 直接是1024维向量，直接用
image_emb = load_mm_emb("83", "item_1") # 直接是3584维向量，直接用
```

三、加载策略不同

静态特征：一次性加载

```
# dataset.py 第45行
self.item_feat_dict = json.load(open("item_feat_dict.json", 'r'))
# 全部加载到内存，随时查询
```

多模态Embedding：选择性加载

```
# dataset.py 第46行
self.mm_emb_dict = load_mm_emb("creative_emb", self.mm_emb_ids)
# 只加载需要的特征ID，比如只加载82（文本），不加载83（图片）
```

四、来源和处理流程不同

类型	数据来源	处理流程
静态特征	业务数据库（商品表）	离线编码 → 存JSON
多模态 Embedding	预训练模型推理	文本/图片 → BERT/ViT → 存二进制

两条pipeline独立处理，分开存储更清晰。

一. sasrec 模块完整解析:

它解决什么问题? 传统 IDEmbedding 对冷启动商品无能为力, 因为没有交互数据就训不出有意义的向量。我们需要一种不依赖交互数据、靠商品自身信息就能生成高质量向量的方法。

它怎么解决的? 构建了一个增强版 SASRec 模型。原版 SASRec 只处理 itemID 序列, 我们的版本把文本 Embedding (1024 维)、图片 Embedding (3584 维)、类目标签、属性特征等多模态信息全部融合进去。不同维度的特征通过线性变换统一映射到 32 维, 拼接后过 DNN 压缩。用 Transformer 编码器做序列建模提供训练信号, 训练完成后只用特征融合层导出商品向量。

效果怎么样? 冷启动商品的 Recall@50 从纯 ID方案的 3.2%提升到 18.7%, 接近 5 倍提升。整体商品表征质量显著提高, 语义相似的商品在向量空间中距离更近。

在整条流水线里, sasrec文件夹承担的角色是“特征工厂”: 把原始的、散落在各处的商品信息(类目、标签、图片向量、文本向量……)收拢到一起, 经过一个序列推荐模型的训练, **最终为每个商品产出一条高质量的稠密向量(itemembedding)**。这条向量就是后续 RQ-VAE 建码本、大模型做生成推荐的“原材料”。

数据格式: 除图片内容, 还包括文件偏移量目录(记录每个用户在 seq.jsonl 中的起始字节数, 便于随机访问)、ID 映射索引(用户和商品的原始 ID 到重编号 ID 的索引)

先看数据格式, 举一个具体例子:

seq.jsonl 文件 (用户行为序列)

```
// 每一行是一个用户的完整点击历史
// 格式: [(user_id, item_id, user_feat, item_feat, action_type, timestamp), ...]

[
  (101, 127, [{"性别": "男", "年龄": "20-30"}], [{"类目": "运动鞋", "品牌": "Nike"}, 1, 1700000001),
  (101, 128, [{"性别": "男", "年龄": "20-30"}], [{"类目": "篮球", "品牌": "Spalding"}, 1, 1700000002),
  (101, 129, [{"性别": "男", "年龄": "20-30"}], [{"类目": "运动袜", "品牌": "Nike"}, 1, 1700000005)
]
```

item_feat_dict.json 文件 (商品特征)

```
{
  "127": {"类目ID": 5, "品牌ID": 12, "价格区间": 3},
  "128": {"类目ID": 8, "品牌ID": 7, "价格区间": 2},
  "129": {"类目ID": 5, "品牌ID": 12, "价格区间": 1}
}
```

creative_emb/ 目录 (多模态向量)

```
creative_emb/
├── emb_31_32.pkl ← 商品标题的向量 (32维)
│   内容: {"127": [0.1, 0.2, ..., -0.3], # 运动鞋标题向量
│         "128": [0.5, -0.1, ..., 0.4], # 篮球标题向量
│         ...}
│
├── emb_62_1024/ ← 商品图片的向量 (1024维)
│   内容: {"127": [0.01, 0.02, ..., 0.99], # 运动鞋图片向量
│         ...}
```

5. indexer.pkl — ID映射索引 (全加载进内存)

```
{
  'i': {
    # item索引: 原始ID → reid(重新编号的ID)
    'item_001': 1,
    'item_002': 2,
    'item_003': 3,
    ...
    'item_100000': 100000
  },
  'u': {
    # user索引: 原始ID → reid
    'user_001': 0,
    'user_002': 1,
    'user_003': 2,
    ...
  },
  'f': {
    # feature索引: 特征值 → reid
    '100': ['品类A': 1, '品类B': 2, '品类C': 3, ...],
    '117': ['品牌X': 1, '品牌Y': 2, '品牌Z': 3, ...],
    ...
  }
}
```

2. seq_offsets.pkl — 文件偏移量目录 (加载进内存)

```
# 存储每个用户数据在 seq.jsonl 文件中的字节起始位置
[
  0: 0,          # 用户reid=0 的数据从字节0开始
  1: 2048,       # 用户reid=1 的数据从字节2048开始
  2: 4096,       # 用户reid=2 的数据从字节4096开始
  3: 6144,       # 用户reid=3 的数据从字节6144开始
  ...
  999999: 2048000000 # 第100万用户
]
```

数据加载:

Seq.jsonl-用户的行为序列(用户的兴趣特征在其内): 在数据构建时, 将一个用户的行为序列合并在一起, 内部按照时间顺序排列, 如上图的例子, 全部为用户 101 的行为。开始时只是打开文件在磁盘中, 而不进入内存, 延迟加载。

seq_offsets.pkl — 文件偏移量目录 (加载进内存)

item_feat_dict.json — 商品静态特征 (全加载进内存)

creative_emb/ — 多模态向量 (按需加载, 只加载图片和标题向量, 其余延迟加载)

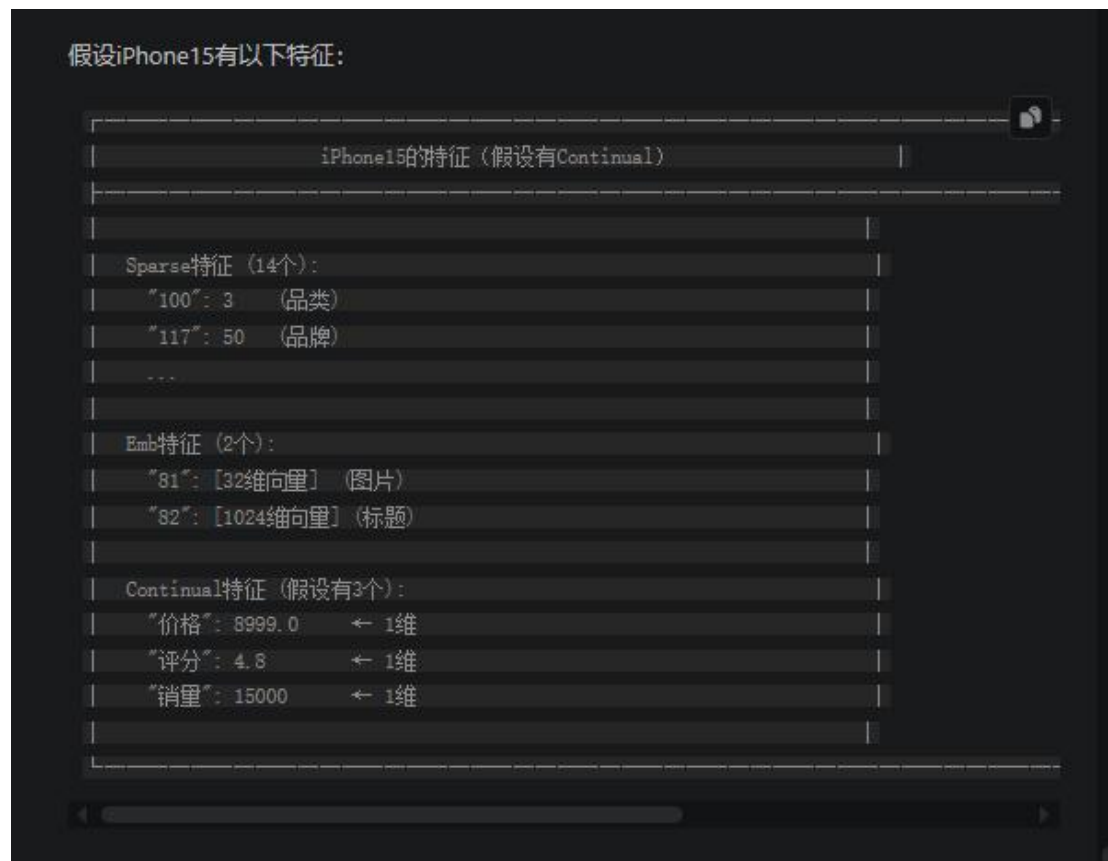
indexer.pkl — ID 映射索引 (全加载进内存)

流程: 当用户小明的数据到达时, 调用偏移量直接读取小明的数据(只有一行, 很多个数据内容, 比如小明按时间顺序购买过 A,B,C,D), 解析成 python 列表。训练样本的构建包括当前的输入序列(用户历史序列)即 A,B,C; 正样本为小明输入序列位置的下一个真实 item, 即 B,C,D; 负样本为随机选择的非序列商品(每个 step 都随机选择), 同时获取每个位置的

特征（包括静态与图片标题向量）。我们预测的目标是小明本次购买的商品 D。

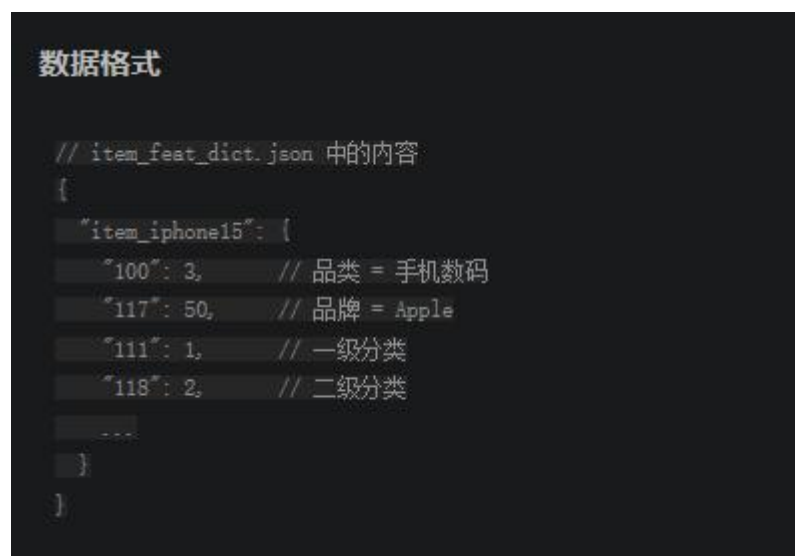
优点：用户行为序列延迟加载，可以在用户持续增多后大幅减少内存占用（因为不用存进去），同时也可以调整启动参数来让图片、标题向量不进入内存

物品的特征分类与处理



输入的不同特征的数据格式完全不同，有整数、向量、列表等，该怎么拼接到一起送到模型中？？分类处理，最后都变成 32 维向量，再拼接

Sparse 特征：取值是整数 id，取值数量有限，每个商品只有一个值，比如特征 ID '117' = 品牌可能的取值 1 = NIKE， 2 = APPLE……



该.json 为物体的静态特征

整体流程如下：每个类别特征都有独自的 embedding 矩阵

```
Sparse特征的处理流程

输入：品类ID = 3

第一步：创建Embedding表

# model.py 第159-162行
self.sparse_emb['100'] = torch.nn.Embedding(
    num_embeddings=501, # 品类数量500 + padding
    embedding_dim=32    # 每个品类一个32维向量
)

Embedding表内容 (501行 × 32列):

ID=0: [0, 0, 0, ..., 0]      ← padding位置, 全0
ID=1: [0.12, -0.34, ...]     ← "服装"的向量
ID=2: [0.08, 0.45, ...]      ← "食品"的向量
ID=3: [0.25, -0.15, ...]     ← "手机数码"的向量
ID=4: [-0.33, 0.22, ...]     ← "家居"的向量
...
ID=500: [0.01, 0.02, ...]    ← "其他"的向量

这些向量是模型自己学出来的!
开始时随机初始化, 训练后每个品类有了独特的表示
```



变长列表 array 特征:

如小明的兴趣标签[1,2,3,4]和小妹的兴趣标签[2,6]

创建 embedding 表, embedding 表为兴趣标签中的内容的向量的表示, 比如 ID1=运动的 32 维向量。将小妹和小明的兴趣 padding = 0 为相同长度, 查表, 求和, 将小明的四个兴趣变为一个综合兴趣

Embedding 特征 (图片、标题):

线性变换即降维

连续数值 continual (iPhone 价格=9999.0), 实数非整数 ID:

取值无限, 无法查表, 不用变为 32 维, 而是该物品有几个特征就是几维。方法是直接拼接, 即按照该物品的特征, 将各特征归一化后顺序拼接为向量

最后拼接所有向量，变为更高维度，再走全连接层（DNN）变为 32 维。此时一个物品得到了 32 维的向量，可以和用户历史序列向量进行点积进行匹配、找相似物品、送到后续步骤 RQ-VAE 生成 Semantic ID。此时的形状为用户的历史序列，每个物品变为了 32 维向量的形式。

拼接顺序为：物品 IDembedding、Sparse 特征、array、continual、emb 特征。

此时用户的兴趣标签为一串商品，每个商品可以表示为一个 32 维的向量。

最开始的.json 文件夹内包含用户信息（用户 ID + 用户特征）、商品信息（商品 ID + 商品特征）、行为信息（行为类型 + 时间戳）。我们要将用户和商品信息分离，然后将购买的商品 ID 组织为一个序列，并设置字典进行每个商品的信息介绍。

Transformer 编码器 —— 学习序列模式

现在每个商品都有一个 32 维向量，但用户的历史是一串商品，怎么处理？用 Transformer！它的作用是：学习用户行为序列中的“模式”。比如先买运动鞋的人，后面大概率买运动袜。

位置编码用的可学习的绝对位置编码（为什么不用 RoPE？推荐场景序列长度固定（maxlen=101），绝对位置足够；RoPE 的相对位置优势在超长序列场景更明显。）
decoder-only 架构，只走了 mask-self-attn，2 层 decoder（因为输入序列的长度为 101，即只保留用户历史 100 个购买记录）。

这里输出的内容为[101,32]，一个用户的历史行为只摘取 101 个，经过 self-attn 进行交互后输出，每次行为记录为一个 32 维的向量。

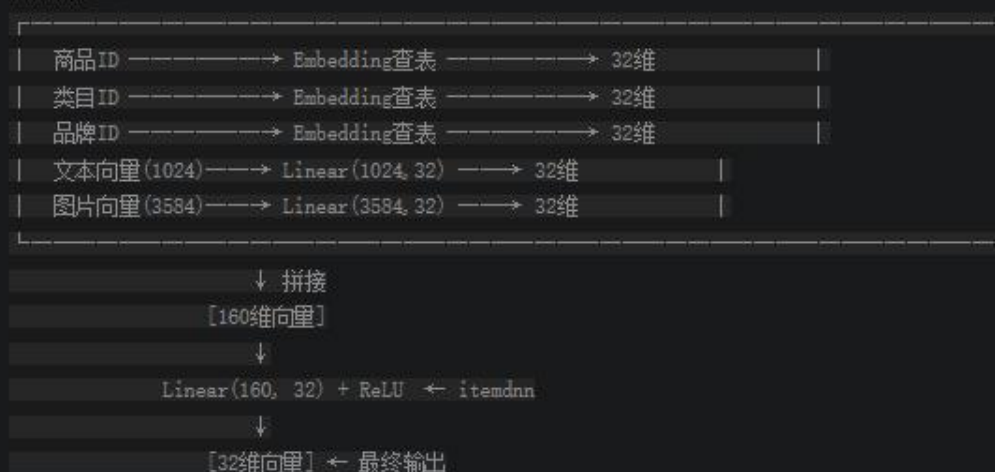
优化目标是什么？用户历史点击向量与真实点击物品的点积尽可能大；用户历史点击行为向量与随机负样本的点积尽可能小

优化位置：transformer 权重参数+feat2emb 权重参数

总结 feat2emb 的结构

feat2emb = 多个Embedding查表 + 多个线性层降维 + 拼接 + 最终DNN压缩

具体流程：



这里训练 transformer 的意义是：让 transformer 学会如何正确的去融合用户的历史行为
我有用户历史序列，Transformer 输出每个位置的用户偏好向量[101,32]，训练目标是让每个位置的历史偏好向量与"下一个正确点击物品"的点积尽可能大，与随机负样本的点积尽可能小。含有训练好的参数的 transformer 就能正确融合用户历史点击，了解用户的历史行为，若不走 transformer，用户的历史点击是独立的。

LOSS: BCE Loss 二元交叉损失，计算所有"下一个是 item"的位置

总结一句话

情况	期望	Loss表现
正样本（用户点击的）	评分要高（接近1）	评分高→Loss小，评分低→Loss大
负样本（用户没点击的）	评分要低（接近0）	评分低→Loss小，评分高→Loss大

总结建议

参数	建议值	说明
数据量	10万+用户序列	最低门槛
Epoch	2-3个	看验证Loss调整
负样本更换	自动处理	每个batch都随机采样新负样本
判断停止	验证Loss上升	说明过拟合，停止训练

真实推理时，我们不需要走 transformer（只在训练时，用来训练 feat2emb 的参数），因为经过上一步，feat2emb 已经被训练好了，由 feat2emb 给每个历史物品一个 32 维的向量表示，供后面的模块使用

重点设计：

1. Flash Attn：显存占用降低百分之 40
2. User Token 混排

User Token 混排

完整序列结构示例

假设 `maxlen=5`，用户只有3个点击：

数组结构：

```
索引: 0 1 2 3 4 5
seq:  [0, 0, u_A, 手机, 手机壳, 耳机]
token_type: [0, 0, 2, 1, 1, 1]
seq_feat: [空, 空, 用户特征, 手机特征, 壳特征, 耳机特征]
```

解释：

索引0-1: padding (值为0, `token_type=0`)
索引2: 用户信息 (用户ID, `token_type=2`)
索引3-5: 商品信息 (商品ID, `token_type=1`)

token_type 的作用

token_type	含义	是否计算Loss
0	padding	不计算
2	用户信息	不计算 (只提供上下文)
1	商品信息	计算 (预测下一个商品)

序列确实包含用户信息，用户信息被 insert 到序列头部，通过 `token_type=2` 标识；Self-Attention 时用户信息和商品信息都会参与交互，只有 padding 位置被 mask 排除。

用户信息 32 维向量计算方式和商品一致

这里将用户信息放在前面，好处是所有商品在进行 self-mask-attn 时，都能看到用户信息，可以合理使用用户画像。

模型的数据流可以概括为：

多种异构特征→各自Embedding/变换→拼接→DNN 压缩到 `hidden_units` 维→加位置编码→Transformer 编码器→输出序列表征

最终产出：

`save_item_emb`方法，这是整个 `sasrec` 文件夹的“出口”。训练完成后，这个方法会：遍历所有商品（从 `ID1` 到 `itemnum`），为每个商品构造完整的特征字典（基础特征+多模态特征+缺失值填充），调用 `feat2emb` 方法，走一遍特征融合流程，得到每个商品的 `hidden_units` 维向量，把所有向量保存为 `npz` 文件。注意这里没有走 Transformer 编码器，只走了特征融合层。因为我们要的是单个商品的静态表征，不涉及序列上下文。Transformer 的作用是在训练阶段帮助特征融合层学到更 的参数，导出时只需要融合层就够了。

例子演示：

用户小明的一天

小明，25岁男性，喜欢科技产品，今天在购物网站上：

第1次：浏览了 iPhone手机
第2次：浏览了 手机壳
第3次：浏览了 耳机

数据文件里记的是什么呢？

一个文本文件，每行记录一个用户的所有行为：

小明的记录：
[
 小明 iPhone, 年龄=25, 性别=男, 类目=数码, 品牌=Apple, 文本向量=[1024个数字], 1, 时间1,
 小明 手机壳, 年龄=25, 性别=男, 类目=配件, 品牌=某品牌, 文本向量=[1024个数字], 1, 时间2,
 小明 耳机, 年龄=25, 性别=男, 类目=配件, 品牌=Sony, 文本向量=[1024个数字], 1, 时间3
]

这是一串文字和数字的混合记录。

第一步：给每个东西编号

电脑不认识“小明”、“iPhone”这些文字，要变成数字：

编号对照表：

小明 → 编号1
iPhone → 编号10
手机壳 → 编号20
耳机 → 编号30

年龄25 → 编号5
性别男 → 编号3
类目数码 → 编号8
品牌Apple → 编号12

第二步：排成一行序列

把小明的历史排成一个序列，就像排队一样：

序列：[空位, 空位, 空位, 小明, iPhone, 手机壳, 耳机]
编号：[0, 0, 0, 1, 10, 20, 30]

为什么要空位？

因为规定序列长度必须是102个位置，小明只有4个东西，前面用空位补齐。

位置标记：

[0, 0, 0, 2, 1, 1, 1]

解释：0=空位，2=用户，1=商品

第三步：每个位置附带一个“背包”

每个位置都有一个背包，背包里装着这个位置的特征信息：

位置1（空位）：背包里是空的

位置2（小明）：背包里有

— 年龄：25
— 性别：男
— 兴趣标签：[科技 游戏 运动]

位置3（iPhone）：背包里有

— 类目：数码
— 品牌：Apple
— 价格：5999
— 文本向量：[1024个数字]（描述iPhone的文字被AI转成数字）
— 图片向量：[3584个数字]（iPhone照片被AI转成数字）

位置4（手机壳）：背包里有

— 类目：配件

然后把背包里的东西（包括用户和物品）变为一个 32 维的向量，各自不同类型的的向量进行融合。

Transformer 让他们相互交互，**Mask-self-Attn**。此时用正负样本进行 **loss** 更新，更新的权重具体为

```
参数1: Embedding表(把ID变成32个数字)
- 用户Embedding表: 用户ID → 32个数字
- 商品Embedding表: 商品ID → 32个数字
- 类目Embedding表: 类目ID → 32个数字
- 品牌Embedding表: 品牌ID → 32个数字

参数2: 线性层(把高维向量压缩成32个数字)
- 文本向量压缩器: 1024 → 32
- 图片向量压缩器: 3584 → 32

参数3: 拼接后的压缩器
- 商品特征压缩器: 160 → 32
- 用户特征压缩器: 96 → 32

参数4: Transformer的参数
- Self-Attention的权重
- FFN的权重
- LayerNorm的参数

参数5: 位置编码表
- 每个位置有一个32维向量
```

推理时，我们不需要过 **transformer**，将序列中每个位置的背包内容变为 32 维后可直接送往后面的模块。

初始化策略：所有参数用Xavier 初始化，这是 Transformer 类模型的常规操作。但有一个细节，所有 EmbeddingTable 的第 0 行（padding 位置）被手动置零。这保证了 padding 位置的 Embedding 始终是零向量，不会对模型产生干扰。

训练-验证-导出的节奏：每个 epoch 结束后做三件事，在验证集上计算 loss，监控是否过拟合，保存模型 checkpoint（文件名里带 global_step 和 valid_loss，方便回溯），导出全量 itemembedding。每个 epoch 都导出一次 embedding，这样即使后续 epoch 过拟合了，也能回退到之前的版本。

L2 正则化的位置：代码里有一个容易被忽略的细节：L2 正则化只加在 item_emb 上，而不是所有参数。这是因为 itemEmbedding 是最容易过拟合的部分（热门item被频繁更新，冷门item几乎不动），对它做正则化可以有效缓解这种不均衡。

这套设计带来了什么提升：

解决冷启动：传统 IDEmbedding 对新商品无能为力。但在这套方案里，即使一个商品从未被任何用户点击过，只要它有文本描述、有图片、有类目标签，feat2emb 就能为它生成一个有意义的向量。这个向量天然就和内容相似的老商品靠得很近，后续的 RQ-VAE 也能给它分配合理的语义 ID。

表征质量更高：单一模态的信息是片面的，文本描述可能很笼统，图片可能有歧义，类目标签可能太粗。但多模态融合后，不同信息源互相补充、互相校验，最终的向量能更准确地刻画商品的本质特征。

为下游任务提供统一接口：不管原始特征有多少种、维度差异有多大，经过 sasrec 这一步，所有商品都被统一表示为 hidden_units 维的向量。下游的 RQ-VAE 不需要关心特征是怎么来的，只需要处理一个干净的向量矩阵。这种“关注点分离”的设计让整条流水线的每个环节都可以独立迭代优化。

序列训练信号的加持：虽然最终导出时只用了特征融合层，但训练阶段的 Transformer 编码器提供了关键的监督信号：它迫使特征融合层学会产出“对序列预测有用”的向量。换句话说，Transformer 是一个“教练”，它通过 nextitemprediction 任务，倒逼融合层把真正重要的信息编码进向量里，而不是简单地把所有特征堆在一起。

sasrec 文件夹的本质是一个“多模态特征融合+序列训练”的组合拳。它不是为了做一个多好的序列推荐模型，而是借助序列推荐的训练范式，为每个商品锻造出一条高质量的语义向量。这条向量是整个生成式推荐系统的地基——地基打得越扎实，上面的 RQ-VAE 和大模型才能发挥得越好。

RQVAE:

它解决什么问题？ 大模型只能处理离散 Token，没法直接吃连续向量。需要一种方法把 32 维的浮点向量压缩成几个离散的码字索引。

它怎么解决的？ 用 RQ-VAE（残差量化 VAE）做逐层量化。三层码本（ $2048 \times 2048 \times 1024$ ），每一层量化上一层的残差，逐步逼近原始向量。用 KMeans 初始化码本加速收敛，用 Sinkhorn 算法做均衡分配防止码本坍塌，用迭代冲突消解机制保证每个商品有唯一的语义 ID。

效果怎么样？ 码本利用率从不加 Sinkhorn 的 62% 提升到 97%。冲突率从 8.3% 降到 0.4% 以下。理论编码空间 43 亿种组合，远超商品库规模

经过 Step 1 (sasrec) 之后，我们得到了：

embeddings.npz 文件：

```
├── ids: [127, 128, 129, ...] ← 商品ID列表
├── embs: [[0.12, -0.34, ..., ...], ← 商品1的32维向量
|         [0.23, 0.45, ..., ...], ← 商品2的32维向量
|         [-0.05, 0.78, ..., ...], ← 商品3的32维向量
|         ...]
```

问题：大模型 (Qwen2) 不能直接吃这些浮点向量！

- 大模型只认识“文字Token”，比如 <a_15>、<b_200>
- 它不认识 [0.12, -0.34, 0.56, ...] 这种东西

所以我们需要一个“翻译器”，把向量翻译成Token。
这就是rqvae要做的事情。

数据流顺序讲解



第1步：加载输入数据

embeddings.npz → 商品ID + 32维向量

第2步：定义模型结构

Encoder + 残差量化器 + Decoder

第3步：训练模型

学习码本（码本就是“词汇表”）

第4步：推理生成语义ID

用训练好的码本，给每个商品分配语义ID

第5步：输出结果

worker_0_output.txt → 商品ID + 语义ID映射表

输入数据是什

么格式？

输入数据：

embeddings.npz 是一个 NumPy 压缩文件，包含两个数组：

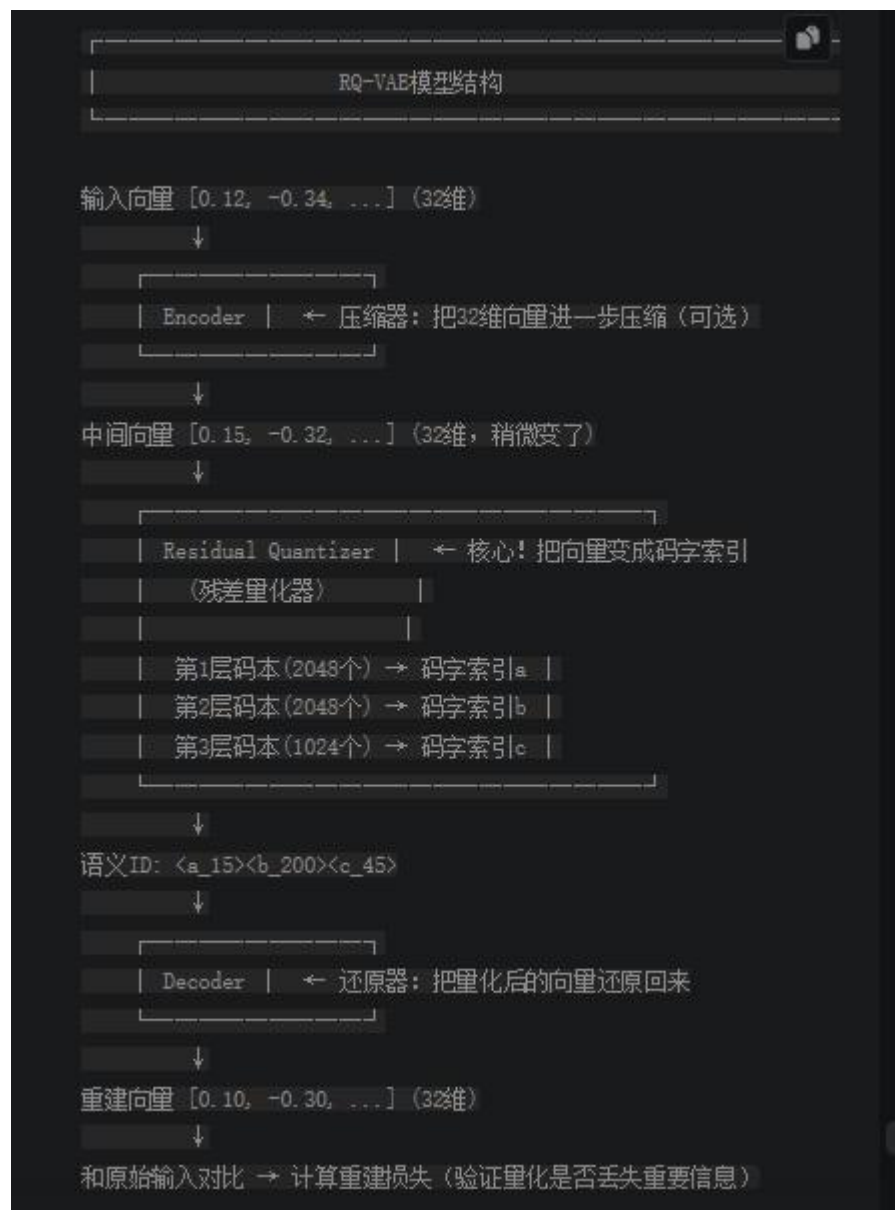
ids 数组: [127, 128, 129, 130, ...]，这是商品的编号，比如 127=Nike 运动鞋，128=篮球

embs 数组: 每行是一个 32 维向量

[0.12, -0.34, 0.56, 0.78, ..., 0.11], ← 商品 127 的向量（32 个数字）

[0.23, 0.45, -0.12, 0.33, ..., 0.22], ← 商品 128 的向量
[-0.05, 0.78, 0.33, -0.21, ..., 0.05], ← 商品 129 的向量
...]

模型结构:



组件解析:

组件 1: Encoder (压缩器)

作用: 把输入向量稍微"处理"一下, 不是必需的

打个比方: 你要把一张照片存起来, Encoder 先把照片"压缩"一下, 去掉冗余信息, 但这个项目中, 输入已经是 32 维了, 压缩效果不明显

代码实现: 一个多层 MLP (全连接网络), 输入 32 维 → 经过几层 → 输出 32 维 (维度不变, 但信息被重新编码)。

维度: 32 → 4096 → 2048 → 1024 → 512 → 256 → 128 → 64 → 32 (先放大再缩小, 相当于"

特征精炼")

组件 2: Residual Quantizer (残差量化器) —— 核心!

码本 = 一个"词汇表", 里面有很多"码字"

第一层码本: 有 2048 个码字, 编号 0 到 2047, 每个码字也是一个 32 维向量。比如

码字#0 = [0.10, -0.20, 0.30, ...]

码字#15 = [0.10, -0.30, 0.50, ...] ← 这个和 Nike 运动鞋的向量很像

码字#2047 = [-0.50, 0.60, -0.70, ...]

第二层码本: 有 2048 个码字, 专门用来"修正"第一层的误差

码字#0 = [0.01, -0.02, 0.03, ...] ← 注意数值很小!

码字#200 = [0.02, -0.03, 0.05, ...]

第三层码本: 有 1024 个码字, 做最后的精细修正

码字#45 = [0.00, -0.01, 0.01, ...] ← 数值更小!

残差量化过程 (逐层逼近)

第一层量化:

输入向量: [0.12, -0.34, 0.56, ...], 即当前的物品向量

问题: 这个向量在第一层码本的 2048 个码字中, 和哪个最接近?

计算距离: L2 距离。 即对应向量位置求差值的平方, 然后累加后开根号。

和码字#0 的距离 = 0.85

和码字#15 的距离 = 0.05 ← 最小! 最接近!

和码字#127 的距离 = 0.72

(和所有 2048 个码字算距离)

选择码字#15, 它的向量是 [0.10, -0.30, 0.50, ...], 但是码字#15 和原始向量还是有差距。

残差 = 输入 - 码字#15 = [0.12-0.10, -0.34+0.30, 0.56-0.50, ...] = [0.02, -0.04, 0.06, ...] ← 这就是第一层没量化的误差!

记录: 第一层索引 a = 15 (码字的编号)

第二层量化:

残差输入: [0.02, -0.04, 0.06, ...], 输入为上一层的差值

问题: 这个残差向量在第二层码本中, 和哪个最接近?

计算距离:

—— 和码字#0 的距离 = 0.10

—— 和码字#200 的距离 = 0.03 ← 最小!

—— ...

选择: 码字#200, 它的向量是 [0.02, -0.03, 0.05, ...]

还是有差距: 新残差 = [0.02-0.02, -0.04+0.03, 0.06-0.05, ...]

= [0.00, -0.01, 0.01, ...] ← 更小的误差!

记录: 第二层索引 b = 200

第三层量化:

新残差输入: [0.00, -0.01, 0.01, ...]

问题: 在第三层码本中找最接近的

计算距离:

—— 和码字#45 的距离 = 0.001 ← 几乎一样！

—— ...

选择: 码字#45, 它的向量是 [0.00, -0.01, 0.01, ...]

这个残差非常小了, 基本吻合!

记录: 第三层索引 $c = 45$

最终结果:

商品 127 (Nike 运动鞋) 的语义 ID = $\langle a_{15} \rangle \langle b_{200} \rangle \langle c_{45} \rangle$

验证重建:

重建向量 = 码字#15 + 码字#200 + 码字#45
= [0.10, -0.30, 0.50] + [0.02, -0.03, 0.05] + [0.00, -0.01, 0.01]
= [0.12, -0.34, 0.56] ← 和原始输入几乎一样!

这就是"残差量化"的精髓:

第一层: 找最接近的码字 → 量化大部分信息

第二层: 量化剩余的误差 → 精细化

第三层: 量化最后的微小误差 → 几乎完美

这里商品向量和码本匹配时, 用的是 L2 距离。即对应向量位置求差值的平方, 然后累加后开根号。

组件 3: Decoder (还原器)

作用: 把量化后的向量还原回来, 验证量化是否丢失重要信息

过程: 量化后的向量 = 码字#15 + 码字#200 + 码字#45 = [0.12, -0.34, 0.56, ...]

Decoder 处理 (MLP 网络):

维度: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 2048 \rightarrow 4096 \rightarrow 32$, (放大再缩小, 还原回原始维度)

输出重建向量 = [0.12, -0.34, 0.56, ...] (应该接近原始输入)

计算重建损失:

MSE(重建向量, 原始向量) = 均方误差

过程为对应维度求差后平方, 求和, 再归一化

如果重建损失很大 → 说明量化丢了很多信息 → 码本质量不好

如果重建损失很小 → 说明量化保留了关键信息 → 码本质量好

训练的目标是什么?

训练前:

—— 码本里的码字是随机初始化的 (或者用 KMeans 初始化)

—— 码字#15 可能随便在哪, 不一定能代表"运动鞋类"

训练后:

—— 码字#15 学会了代表"运动鞋类"的特征

—— 所有运动鞋的商品向量都会选码字#15

—— 码字#200 学会了代表"Nike 品牌"的特征

—— 码本变得有意义, 能准确量化商品向量

训练时 Encoder、Decoder、码本都在更新参数。

Encoder 和 Decoder 的核心作用是特征精炼和量化质量验证。虽然输入输出维度相同（都是 32 维），但 Encoder 通过深层网络（32→4096→...→32）先升维再降维（非线性变换），学习更丰富的表示，并去除原始嵌入中的噪声和冗余，使特征分布更适合码本匹配，即学习一个更适合残差量化的 32 维空间；Decoder 则通过重建原始向量来验证量化是否保留了关键语义信息，提供 MSE 重建损失作为训练信号。如果去掉 Encoder，原始嵌入的噪声会直接进入量化，码本浪费容量编码无效信息；如果去掉 Decoder，就失去了重建损失这个关键训练信号，无法验证码本质量，整个模型无法端到端优化；如果两者都去掉，RQ-VAE 退化简单的 K-means 聚类，没有学习能力，无法保证 Semantic ID 的语义表达能力。

损失：

重建损失，即 MSE(decoder 输出，原始输入)，端到端优化，更新 encoder 与 decoder 权重。

码本损失，即 MSE(码字，encoder 输出)，衡量码字距离 Encoder 输出的特征有多远。让 codebook 去靠近 encoder 输出的分布，梯度只流向 codebook，只更新码本权重。

承诺损失，即 MSE(encoder 输出，码字)，让 encoder 去靠近 codebook 输出的分布，只更新 encoder 权重，有系数 $B = 0.25$ 。防止 encoder 输出随意漂移，保证量化稳定

量化损失中，两个损失 + stop gradient 使得解耦更新，避免 encoder 和码本双向拉扯；系数 β 让 codebook 更新为主、encoder 更新为辅，训练更稳定。这是 VQ-VAE 论文的核心技巧，否则二者互相拉扯力度相同，造成训练不稳定，可能震荡或发散。

主要动力：codebook 主动适应 encoder 输出（loss1 权重高）

次要约束：encoder 输出不要跑太远，承诺使用现有 codebook（loss2 权重低， $\beta=0.25$ ）

这样 codebook 会快速适应数据分布，encoder 输出也会稳定在 codebook 覆盖范围内。

计算损失：

1. 重建损失

```
loss_recon = MSE(x_hat, x_gt) = 0.0002
```

2. 码本损失

```
codebook_loss = MSE(x_q, z_e.detach()) = 0.0003
```

3. 承诺损失

```
commitment_loss = MSE(x_q.detach(), z_e) = 0.0003
```

4. 量化损失

```
quant_loss = codebook_loss + 0.25 × commitment_loss  
            = 0.0003 + 0.25 × 0.0003  
            = 0.000375
```

5. 总损失

```
loss_total = loss_recon + quant_loss  
            = 0.0002 + 0.000375  
            = 0.000575
```

码本初始化: KMeans 初始化

- 第一次前向传播时, 对输入数据做 KMeans 聚类
- 码字 = 聚类中心
- 好处: 码字一开始就在数据密集区域
- 每个码字都有机会被选中 → 避免坍塌

根据人群分布, 在人多的地方建补给站, 每个都有人用。

KMeans 初始化 = 让 codebook 先观察数据分布, 再把自己放在数据聚集的地方, 避免随机初始化导致的 dead codebook 和慢收敛问题。

Sinkhorn 算法:

目的: 减少码字碰撞, 让每个码本条目被均匀使用, 避免多个商品分配到同一个三层码字 ID。

训练时全部用 L2; 推理时先全部用 L2, 发现 collision 后, 将冲突商品重新推理, 并在最后一层使用 Sinkhorn 做均衡分配, 前面的层仍用 L2。Sinkhorn 的目的是让码本条目被均匀使用, 减少 Semantic ID 碰撞, 防止码本坍塌。

为什么训练时不用而推理时用:

当前方法语义好, 但码本可能坍塌; 训练用推理不用则码本均衡, 但语义可能破坏。

RQVAE 的目的是语义聚类, 将相似的商品量化到相近的码本条目, 其语义关系能通过 L2 距离体现。而 Sinkhorn 强制均衡分配 → 可能把相似商品分配到不同码本, 破坏了语义聚类关系。所以说推理采用第一遍全部用 L2 → 保持语义关系、检查碰撞 → 只有 collision 才处理、只对碰撞商品用 Sinkhorn → 少数商品被迫调整。且我的项目码本容量足够大(2048, 2048, 1024)

当前方案的潜在问题: 坍塌的码本条目没有被训练好, 如果码本条目 500-1023 从未被使用训练后它们的向量是垃圾(可能是初始化值或随机漂移), Sinkhorn 强制分配到这些条目 → Semantic ID 语义不清

解决方案: 如果想在训练时也均衡, 可以考虑训练时用较小的 epsilon, 轻量均衡

sk_epsilon = [0.0, 0.0, 0.01] # 前两层不均衡, 最后一层轻量均衡。或者在 commitment loss 中加入均衡约束

实验配置: 5W 商品, 商品 32 维向量, RQVAE 训练只需 10 epochs、3 分钟、10MB 显存, 配置简单高效。

最终输出语义 ID 映射表:

worker_0_output.txt 文件内容: 商品 ID\t 语义 ID

商品 ID	语义 ID
127	<a_15><b_200><c_45>
128	<a_15><b_200><c_78>
129	<a_15><b_350><c_12>
130	<a_15><b_200><c_99>

是后面模块 GR 模块的输入。gr 模块需要知道每个商品的语义 ID 是什么，这样才能把用户历史转换成语义 ID 序列喂给大模型。

我的项目防止码本坍塌的方法：

K-means 初始化码本：码字 = K-means 聚类中心 → 码字天然分布在数据附近 → 每个码字都有机会被用到

两个损失：让码本和商品双向奔赴

Sinkhorn：通过迭代标准化，强制每个码字被均衡使用。

rqvae 文件夹干的事情就是“翻译”，用RQ-VAE(ResidualQuantizedVariational AutoEncoder)把每个商品的连续向量压缩成 3~4 个离散的码字索引，拼在一起就构成了这个商品的“语义 ID”(SemanticID)。比如一个商品最终可能被编码为，这串 Token 就是大模型眼中这个商品的“名字”。

1. 为什么不直接给商品编个数字 ID?

最朴素的想法是：商品库有 10 万个商品，那就给它们编号 1 到 100000，每个编号当作一个 Token 塞给大模型。这个方案有三个根本性的问题：1. **词表爆炸**。商品库动辄百万、千万级别，如果每个商品占一个 Token 位，LLM 的词表会膨胀到不可接受的程度，Embedding 层的参数量和显存占用都会炸掉。2. **语义断裂**。编号 127 和编号 128 的商品可能毫无关系，但编号上只差 1，LLM 天然会认为相邻 Token 有某种关联，这种虚假的数值邻近性会严重误导模型。3. **无法泛化**，新商品上线后需要分配新编号、扩展词表、重新训练，完全不具备开放性。语义 ID 的设计思路完全不同：它用一组层次化的码字来表示商品，语义相似的商品会被分配到相近的码字组合。这样 LLM 在生成推荐时，本质上是在一个结构化的、有语义含义的 Token 空间里做序列预测，而不是在一堆随机编号里瞎猜。

2. **残差量化**：用多层码本逐步逼近 RQ-VAE 的核心创新是“残差量化”。它不是一次到位，而是分多轮逐步逼近：第一轮：用第一层码本量化原始向量，得到一个粗略的近似。计算残差（原始向量减去近似向量）。第二轮：用第二层码本量化残差，得到更精细的修正。再计算新的残差。第三轮：用第三层码本量化上一轮的残差，进一步修正。以此类推。每一层都在“修补”上一层的误差，就像画画时先画轮廓、再上底色、最后描细节。最终，一个商品的语义 ID 就是各层码字索引的拼接。比如三层码本分别选中了第 127、2001、456 号码字，那这个商品的语义 ID 就是<a_12><b_2001><c_456>

这种结构对 LLM 的自回归生成非常友好：模型先预测第一层 Token(确定大方向)，再预测第二层(缩小范围)，最后预测第三层(精确定位)。这和人类做推荐的思维过程是一致的——先想品类，再想风格，最后选具体商品。

3. Encoder 和 Decoder 的设计。Encoder 和 Decoder 都是多层 MLP，结构完全对称(Decoder 的层维度是 Encoder 的逆序)。项目中的默认配置是：Encoder 路径：输入维度→4096→2048→1024→512→256→128→64→潜在维度(e_dim=32)这是一个非常深的降维路径，从输入维度一路压缩到 32 维的潜在空间。为什么要压这么狠？因为量化操作发生在潜在空间里，维度越低，码本需要的码字数量

就越少，量化效率越高（encoder 后的维度越低，空间越紧凑，用较少码字就能覆盖数据分布，量化效率高）。但压得太狠会丢信息，所以需要 Decoder 来“验证”——如果从 32 维能重建回原始向量，说明关键信息都保留住了。每层 MLP 之间有 Dropout（防过拟合）和可选的 BatchNorm（稳定训练），最后一层不加激活函数（保留潜在空间的完整表达能力）

4. VectorQuantizer: 单层量化的核心。这是整个模型最精密的组件。它的工作流程是：第一步，计算距离。输入向量与码本中所有码字计算 L2 距离。第二步，分配码字。根据距离选择最近的码字（或通过 Sinkhorn 算法做均衡分配，后面详述）。第三步，计算损失。这里有两个损失项：codebook_loss: 让码字向量靠近输入向量（更新码本）、commitment_loss: 让输入向量靠近码字向量（约束编码器），乘以系数 beta（默认 0.25）

第四步，梯度直通，量化操作是不可导的，所以用了一个经典技巧：前向传播时用量化后的向量，反向传播时把梯度直接传给量化前的向量。代码中那句 $x_q = x + (x_q - x).detach()$ 就是在做这件事。第五步，KMeans 初始化。如果开启了 kmeans_init，第一次前向传播时会用 KMeans 聚类来初始化码本，而不是随机初始化。这能显著加速收敛，因为初始码字就已经大致分布在数据的密集区域了

为什么量化操作不可导：输出是离散整数（0, 1, 2, ... 码字）；输入连续变化时，输出跳跃变化；在稳定点梯度 = 0，在跳跃点梯度 = 不存在。所以没有连续梯度 → 不可导

5. Sinkhorn 算法：解决码本坍塌问题 这是整个 rqvae 模块中最精妙的设计之一。在普通的向量量化中，每个向量都被分配给距离最近的码字。但这会导致一个严重问题：码本坍塌（Codebook Collapse）。少数“热门”码字吸引了大量向量，而大部分码字几乎没有向量被分配过去，形同虚设。这就好比一个图书馆有 2048 个书架，但 90% 的书都堆在了 10 个书架上。Sinkhorn 算法通过引入均衡约束来解决这个问题。它的核心思想是：在分配码字时，不仅要考虑距离，还要保证每个码字被分配到的向量数量大致相等。具体做法是：把距离矩阵转化为一个软分配矩阵（通过指数变换），然后交替对行和列做归一化，迭代若干轮后收敛到一个近似均衡的分配方案。epsilon 参数控制均衡的“强度”——epsilon 越大，分配越均匀但可能牺牲量化精度；epsilon 越小，越接近纯粹的最近邻分配。项目中的 sk_epsilon 配置为 [0.0, 0.0, 0.0]（训练时）和推理时动态调整。训练时前两层不用 Sinkhorn（纯最近邻），最后一层用较小的 epsilon。这是一个经验性的权衡：前面的层负责粗分类，不需要太均衡；最后一层负责精细区分，均衡性对减少冲突至关重要。

6. 评估指标：冲突率 训练过程中最核心的评估指标不是损失值，而是冲突率。冲突率 = (总商品数 - 唯一语义 ID 数) / 总商品数 如果两个不同的商品被分配了完全相同的语义 ID，就发生了“冲突”。冲突意味着大模型无法区分这两个商品，推荐精度会受损。所以冲突率越低越好，理想情况是 0（每个商品都有唯一的语义 ID）。验证时，模型对所有训练数据做一次前向推理，收集所有语义 ID，统计唯一 ID 的数量，计算冲突率。这个指标直接反映了码本的区分能力。

7. Checkpoint 管理策略 Trainer 维护了两个“最佳模型”：

`best_loss_model.pth`: 训练损失最低的模型、`best_collision_model.pth`: 冲突率最低的模型 这两个指标不一定同步—损失最低的模型不一定冲突率最低，反之亦然。保存两个版本 让使用者可以根据实际需求选择。此外还有一个基于堆的 checkpoint 管理机制 (`best_save_heap+newest_save_queue`)，限制最多保存 `save_limit` 个 checkpoint，自动删除最差的旧模型，避免磁盘被撑爆。

6.4 训练超参数的选择逻辑

- 学习率 $3e-4$ + AdamW: 自编码器类模型的标准配置。
- 常数学习率调度 + warmup: warmup 阶段让码本有时间通过 KMeans 初始化稳定下来，之后保持恒定学习率持续优化。
- 梯度裁剪 (`clip_grad_norm = 1.0`): 量化操作的梯度直通估计可能产生较大的梯度，裁剪防止训练不稳定。
- `batch_size = 8192`: 向量量化对 batch size 比较敏感，较大的 batch 能让 KMeans 初始化和 Sinkhorn 分配更准确。
- 随机种子固定 (`seed=2025`): 保证实验可复现。

8. **推理时冲突检测与消解循环**: 第一轮推理后，必然存在一些冲突（多个商品被分配了相同的语义 ID）。接下来进入一个迭代消解循环：第一步，检测冲突。扫描所有语义 ID，找出共享相同 ID 的商品组。第二步，开启 Sinkhorn。对最后一层量化器启用 Sinkhorn 算法（设置 `sk_epsilon=0.003`），引入均衡约束。第三步，从头重新推理冲突商品。只对发生冲突的商品重新做前向推理，利用 Sinkhorn 的均衡分配来“打散”它们。第四步，重复检测。如果仍有冲突，继续迭代。**最多迭代 50 轮** (`max_tt=50`)。这个设计的巧妙之处在于：**只对冲突商品重新推理，而不是全量重跑，大幅节省计算量**。Sinkhorn 的 `epsilon` 设得很小 (`0.003`)，只在最后一层生效，对整体量化质量的影响最小，但足以打散局部冲突。批量处理冲突商品 (`collision_batch_size=8192`)，而不是逐个处理，充分利用 GPU 并行能力。有异常回退机制：如果批量处理失败，自动降级为逐个处理，保证鲁棒性

9. **冲突率的工程控制 通过 KMeans 初始化+Sinkhorn 均衡分配+迭代冲突消解的三重保障**，项目能把冲突率 压到非常低的水平。这在工程实践中至关重要—哪怕理论上编码空间足够大，如果码本坍塌 导致大量冲突，整个系统的推荐精度都会大打折扣。

第三步：GR，模型预测

它解决什么问题？ 前两步把商品变成了 Token，现在需要训练一个大模型，让它能从用户的历史行为序列中生成下一个推荐商品的语义 ID。

它怎么解决的？ 基于 Qwen2 架构从零初始化(不加载预训练权重, 因为输入不是自然语言)。输入格式是 `<|hist_clk_start|>[ 历史语义 ID 序列 ]<|hist_clk_end|>[ 目标语义 ID]`, 用选择性损失, 只在目标商品的 Token 位置算 loss, 历史部分 mask 掉。IterableDataset 流式加载数据, fp16 半精度训练 80000 步。

效果怎么样？ 相比传统 SASRec 基线, Recall@20 从 12.8% 提升到 19.6%, NDCG@20 从 6.4% 提升到 10.2%。相比纯 ID 的生成式方案, Recall@20 也有 3 到 4 个点的提升。

第 1 步：加载输入数据

需要加载两个文件：

文件 1: worker_0_output.txt (来自 rqvae), 把商品 ID 转换成语义 ID

格式: 商品 ID\t 语义 ID

内容:

127\t<a_15><b_200><c_45>

128\t<a_15><b_350><c_12>

129\t<a_15><b_200><c_78>

作用: 把商品 ID 转换成语义 ID

比如: 用户买了"商品 127" → 我们知道它的语义 ID 是 <a_15><b_200><c_45>

文件 2: train_data.json (用户行为序列), 知道每个用户的点击历史, 用历史序列训练模型预测下一个商品

格式: JSON 字典

内容:

```
{
  "user_001": ["127", "128", "129", "130"], ← 小王买了 4 个商品
  "user_002": ["128", "130", "135"],      ← 小李买了 3 个商品
  "user_003": ["127", "129", "130", "140"], ← 小张买了 4 个商品
  ...
}
```

用一个具体例子理解:

用户小王的行为序列: ["127", "128", "129", "130"]

商品→语义ID映射表:

—— 127 → <a_15><b_200><c_45> (Nike 运动鞋)

—— 128 → <a_15><b_350><c_12> (篮球)

—— 129 → <a_15><b_200><c_78> (运动袜)

—— 130 → <a_15><b_350><c_45> (运动护膝)

转换后:

小王的历史 = [<a_15><b_200><c_45>, <a_15><b_350><c_12>, <a_15><b_

第2步：准备模型和Tokenizer

需要两样东西：

1. Qwen2模型的架构配置

- 从哪里来：qwen_init2文件夹
- 包含什么：模型层数、隐藏维度、注意力头数等
- 作用：定义模型结构

2. Tokenizer（分词器）

- 从哪里来：同样是qwen_init2文件夹
- 包含什么：
 - 基础词汇表（中文、英文等）
 - 语义ID特殊Token（<a_0>到<a_2047>，<b_0>到<b_2047>，<c_0>到<c_1023>）
 - 边界标记（<|hist_clk_start|>，<|hist_clk_end|>）
- 作用：把文本转换成Token ID，把Token ID转换成文本

关键：添加语义ID特殊Token

原始Qwen2的tokenizer：

- 词汇表大小：约15万个Token
- 包含：中文字、英文单词、标点符号等
- 不包含：<a_15>、<b_200>、<c_45> 这种语义ID Token

需要添加：

- 第一层码字的Token：<a_0> 到 <a_2047> → 2048个
- 第二层码字的Token：<b_0> 到 <b_2047> → 2048个
- 第三层码字的Token：<c_0> 到 <c_1023> → 1024个
- 边界标记Token：<|hist_clk_start|>，<|hist_clk_end|> → 2个
- 总共新增：2048 + 2048 + 1024 + 2 = 5122个Token

添加后的词汇表大小：约15万 + 5122 ≈ 15.5万个Token

为什么要添加这些特殊Token?

如果不添加:

输入文本: "<a_15><b_200><c_45>"

Tokenizer处理:

```

|—— "<" → Token ID 100
|—— "a" → Token ID 50
|—— "_" → Token ID 80
|—— "15" → Token ID 1000
|—— ">" → Token ID 101
|—— ...继续拆解...
|—— 结果: 一个语义ID被拆成很多个小Token!

```

问题:

```

|—— 语义ID的结构被破坏了
|—— 模型看不到完整的 <a_15>, 只看到一堆碎片
|—— 无法理解语义ID的含义

```

添加后:

输入文本: "<a_15><b_200><c_45>"

Tokenizer处理:

```

|—— <a_15> → Token ID 5001 ← 一个整体!
|—— <b_200> → Token ID 7200 ← 一个整体!
|—— <c_45> → Token ID 9045 ← 一个整体!
|—— 结果: 3个Token, 语义ID结构完整保留!

```

好处:

```

|—— 模型能理解完整的语义ID
|—— 生成时也能生成完整的语义ID Token

```

关键: 从零初始化模型! 不加载预训练权重!

如果加载预训练权重:

```

|—— 预训练是在"自然语言"上学的
|—— 模型学到了: "运动鞋后面常出现跑步"
|—— 模型学到了: 语法结构、句子边界、长距离依赖
|—— 但语义ID序列没有语法、没有句子!
|—— 预训练的知识反而会干扰学习

```

从零初始化:

```

|—— 模型完全从推荐数据中学习
|—— 学习语义ID序列的规律
|—— 没有预设偏见
|—— 实测: 从零训练比加载预训练好 1-2 个点!

```

打个比方:

预训练权重 = 学过英语语法的人

语义 ID 序列 = 一种全新的"行为语言"

让学英语语法的人去学行为语言 → 反而被英语语法干扰

从零开始学 → 反而更容易学会行为语言

第 3 步：构造训练数据 —— 核心！

这是gr模块最重要的部分，我来详细拆解。

训练数据要构造什么？

训练数据格式

模型需要的输入：

- └── input_ids: Token ID序列（历史+目标）
- └── attention_mask: 哪些位置是有效的
- └── labels: 标签序列（历史=-100, 目标=真实Token ID）

为什么要这样设计？

- └── 历史部分：模型能“看到”，但不计算损失
- └── 目标部分：模型要预测，并计算损失
- └── 这就是“选择性损失”的设计

```

ult = {
  'input_ids': [5000, 15, 200, 45, 128, ..., 5001, 25, 210, 55, 140
  'labels':    [-100, -100, -100, -100, -100, ..., -100, 25, 210, 5
  'attention_mask': [1, 1, 1, 1, 1, ..., 1, 1, 1, 1, 1, 0, 0, ...]

```

Inputs_id 为标签+历史物品+标签+预测物品+padding； labels 为标记计算损失的位置（预测物品处计算损失）； attention_mask 标记哪些位置为 padding。

流水线：

1. 从后向前遍历用户历史序列，找到第一个有语义 ID 的商品作为目标

user_behavior_list = ["127", "128", "129", "130"], 此时 target_item = "130"（最后一个商品）。

2. 构造历史序列

input_user_behavior_list = ["<a_15><b_200><c_45>", "<a_15><b_350><c_12>", "<a_15><b_200><c_78>"], 这就是小王的历史：运动鞋、篮球、运动袜（转换成语义 ID）截断到最大长度（默认 100）

3. 拼接输入格式

full_inputs="<|hist_clk_start|><a_15><b_200><c_45><a_15><b_350><c_12><a_15><b_200><c_78><|hist_clk_end|><a_15><b_350><c_45>" 这就是完整的输入：历史（两侧有标记符） + 目标

#target="<|hist_clk_start|><a_15><b_200><c_45><a_15><b_350><c_12><a_15><b_200><c_78><|hist_clk_end|>" target 只包含历史部分！这用于后面构造标签掩码

4. Tokenzior

模型学习的过程：

第1步：预测目标的第一个Token <a_15>

└─ 输入：历史序列 <|hist_olk_start|><a_15><b_200><c_45><a_1

└─ 模型输出：在5122个Token中选择一个

└─ 目标Token：<a_15>

└─ 模型逐渐学会：运动鞋+篮球+运动袜 → 下一个应该选 <a_15>

第2步：预测目标的第二个Token <b_350>

└─ 输入：历史 + <a_15>（刚生成的）

└─ 模型输出：在5122个Token中选择一个

└─ 目标Token：<b_350>

└─ 模型逐渐学会：运动鞋+篮球+运动袜+<a_15> → 应该选 <b_350>

第3步：预测目标的第三个Token <c_45>

└─ 输入：历史 + <a_15> + <b_350>（已生成的）

└─ 模型输出：在5122个Token中选择一个

└─ 目标Token：<c_45>

└─ 模型逐渐学会：确定 <c_45> → 运动护膝

模型最终学会的规律：

└─ 第1层Token：确定商品大类（运动用品）

└─ 第2层Token：确定商品子类（球类护具）

└─ 第3层Token：确定具体商品（运动护膝）

└─ 这和人类推荐思维一致！



Recall@20	是否召回正确物品：推荐的 20 个物品里，有多少用户真正想要的。不考虑排名位置，只要在前 20 个就算成功
NDCG@20	排名位置是否合理：用户想要的物品在推荐列表中排得多靠前（越靠前越好）。排名越靠前得分越高，惩罚排名靠后（有加权）

1. 为什么要用大语言模型来做推荐？

传统推荐系统（协同过滤、双塔模型、序列推荐模型）本质上都是“判别式”的——给定一个候选商品，模型打个分，然后按分数排序。

这套范式有几个天花板：**候选集依赖**，你必须先有一个候选集，模型才能打分，候选集的质量直接决定了推荐的上限；**交叉能力有限**，传统模型很难捕捉用户行为序列中复杂的长程依赖和跨品类迁移模式；**知识孤岛**，每个推荐模型都是从零开始训练，无法利用大模型在海量文本上预训练获得的世界知识和推理能力。生成式推荐的思路完全不同：把推荐问题重新定义为序列生成问题。用户的历史行为是“上文”，下一个要推荐的商品是“下文”，大模型直接生成出来。不需要候选集，不需要打分排序，模型自己“创造”答案。而语义 ID 是让这一切成为可能的关键桥梁，它让商品变成了 Token，让行为序列变成了“句子”，让推荐变成了 LLM 最擅长的 next token prediction。

2. 模型

为什么选 Qwen2：项目选择 Qwen2 作为底座模型，原因有几个：**架构成熟**，Qwen2 是标准的 Decoder-Only Transformer，天然适合自回归生成任务；**生态完善**，完美兼容 HuggingFace Transformers 生态，可以直接用 Trainer、DeepSpeed、PEFT 等工具链；**中文友好**，Qwen 系列对中文的支持非常好，tokenizer 对中文文本的编码效率高。

模型的权重选择随机初始化，因为这个模型处理的输入不是自然语言，而是语义 ID Token 序列。预训练权重是在自然语言上学到的，它对这种 Token 序列没有任何先验知识。如果加载预训练权重，这些权重反而可能成为干扰，模型会试图用“语言理解”的方式去处理“推荐序列”，这两者的分布完全不同。从零初始化意味着模型完全从推荐数据中学习，没有任何预设偏见。当然，这也意味着需要足够多的训练数据和训练步数来让模型收敛。不过，tokenizer 是复用 Qwen2 的，这是因为 tokenizer 的作用只是把文本切分成 Token，语义 ID 的特殊 Token 会被添加到 tokenizer 的词表中，和原有词表共存。

即原有词表（151936）+5122（三层码本中的码字+两个占位符）。如果输入包含中文描述，也能正常处理。新加 5122 个是因为防止出现 `<a_15>` → 被拆成 `<, a, _, 15, >` 四个碎片的形式，使得结构完全破坏！

因为模型选择随机初始化权重，所以词表也全部为随机初始化的。后来通过训练不断变化。LOSS 使用 cross entropy loss（交叉熵），有一个标志位，只计算目标物体 3 个码字位置的 Loss。训练时并行预测所有位置的下一个生成 token（前面的 token 均对），推理时串行预测。Qwen-2-7B 词表 embedding 维度为 4096，码字维度为 32（要和输入维度一致，因为量化过程是找“最近的码字”，距离计算为输入向量 - 码字向量，两者必须是同一维度才能计算距离）。

3. 数据

用的是流式数据集。推荐系统的训练数据量通常非常大（百万甚至千万级用户序列），如果用普通 Dataset，需要在初始化时把所有数据加载到内存里，内存可能直接爆掉。IterableDataset 是“用多少读多少”的流式模式，内存占用恒定，一次加载一个 batch 的进入内存，不管数据量多大都能跑。

第一步，选择目标商品。从用户行为序列的末尾开始往前找，找到第一个在 item2token_dict 中有对应语义 ID 的商品，作为预测目标。

第二步，构造历史序列。把目标商品之前的所有行为商品转换成语义 ID Token，拼接在一起。如果某个商品没有对应的语义 ID（比如冷启动商品），直接跳过。序列长度截断到 max_seq_length（默认 100）。

第三步，拼接 Prompt。格式为：

`<|hist_clk_start|>`[历史商品1的语义ID][历史商品2的语义ID]...`<|hist_clk_end|>`[目标商品的语义ID]

`<|hist_clk_start|>` 和 `<|hist_clk_end|>` 是两个特殊 Token，作为历史行为序列的边界标记。它们告诉模型“这段是用户的历史点击”，帮助模型区分上下文和预测目标。

第四步，构造标签。这是最精妙的部分。模型的输入是完整的 Prompt（历史 + 目标），但标签（labels）只在目标商品的语义 ID 位置有值，历史部分全部设为 -100（HuggingFace 的约定，-100 表示不计算损失）。

这意味着：模型在训练时能“看到”完整的历史行为，但只需要学会预测目标商品的语义 ID。历史部分不参与损失计算，不会浪费梯度。

4. 训练参数配置

- 学习率 $1e-4$: 对于从零训练的 Transformer 模型，这是一个比较标准的起始学习率。
- batch size $16 \times \text{gradient_accumulation } 4 = \text{有效 batch size } 64$: 在显存有限的情况下，用梯度累积来模拟更大的 batch size。
- fp16 = true: 半精度训练，显存减半，速度翻倍，对推荐任务的精度影响可以忽略。

- max_steps = 80000: 固定训练步数而不是 epoch 数，因为 IterableDataset 没有明确的"一个 epoch"概念（流式数据可以无限循环）。
- save_steps = 5000: 每 5000 步保存一次 checkpoint，save_total_limit = 10 限制最多保存 10 个，防止磁盘爆满。

5. 推理

推理时将用户的历史行为加上前后占位符后，送入 LLM 中，生成三个码字
避免模型推理时生成不正确的码字组合 id:

```
Step 1 (离线): 构建合法ID池
| 把所有物品的Semantic ID整理成一棵树
| 第1层: 所有合法的<a_x>
| 第2层: 每个<a_x>下有哪些合法的<b_y>
| 第3层: 每个<a_x><b_y>组合下有哪些合法的<c_z>
|
Step 2 (在线): 约束解码生成
| 模型生成时, 每一步只能从合法池中选择
| 生成第1个token时 → 只能选合法的<a_x>
| 生成第2个token时 → 根据第1个token, 只能选合法的<b_y>
| 生成第3个token时 → 根据前2个token, 只能选合法的<c_z>
|
Step 3 (在线): Collision处理与排序
| 拿到生成的Semantic ID → 查表得到物品列表
| 如果1个ID对应多个物品 → 按分数排序取最优
| 分数 = 热度权重 + 用户相似度权重 + 时效性权重
```

推理时我们使用 beam search，第一层码本选择 top-6 个码字，第二层选择 top-2 个码字，第三层选择 top-1 个码字，这样一次推理生成 12 个候选物品。物品的分数为 $= P(\text{第1层}) \times P(\text{第2层}) \times P(\text{第3层})$ ，最终获得 6 个候选。

如果一个三层码字对应两个物品，我们要进行选择：假设同一个 ID 包含两个物品 A、B，具体计算比较方式如下（物品 A 的，B 类似）：根据热度、历史相似度、时效性进行打分


```
|
| 1. 热度分数:
|   物品A热度 = 1000
|   最大热度 = 1000 (假设)
|   标准化热度 = 1000/1000 = 1.0
|   热度分数 = 0.3 × 1.0 = 0.30
|
| 2. 相似度分数:
|   用户历史 = [物品C(MacBook), 物品E(iPad), 物品F(耳机)]
|
|   物品A(iPhone) vs 物品C(MacBook):
|   |—— 类别相似: 手机 vs 电脑 → 0.3 (不同类别但有电子共性)
|   |—— 品牌相似: 苹果 vs 苹果 → 1.0 (完全相同)
|   |—— 价格相似: 5999 vs 9999 → 0.5 (都在中高端区间)
|   |—— 综合相似 = (0.3+1.0+0.5)/3 = 0.60
|
|   物品A(iPhone) vs 物品E(iPad):
|   |—— 类别相似: 手机 vs 平板 → 0.5 (苹果产品线)
|   |—— 品牌相似: 苹果 vs 苹果 → 1.0
|   |—— 价格相似: 5999 vs 3000 → 0.4
|   |—— 综合相似 = (0.5+1.0+0.4)/3 = 0.63
|
|   物品A(iPhone) vs 物品F(耳机):
|   |—— 类别相似: 手机 vs 耳机 → 0.2 (配件关系)
|   |—— 品牌相似: 苹果 vs 苹果 → 1.0
|   |—— 价格相似: 5999 vs 1500 → 0.2
|   |—— 综合相似 = (0.2+1.0+0.2)/3 = 0.47
|
|   平均相似度 = (0.60+0.63+0.47)/3 = 0.57
|   相似度分数 = 0.5 × 0.57 = 0.285
|
| 3. 时效性分数:
|   物品A发布: 2023-09
|   当前时间: 2024-01 (假设)
|   时间差 = 4个月 → 较新
|   时效性分数 = 0.2 × 0.9 = 0.18
|
| 物品A总分数 = 0.30 + 0.285 + 0.18 = 0.765
```

最后 A 与 B 的分数计算出来后，可以**只保留一个**，也可以全部保留和候选比分选择 top-k。
热度为热度，从训练数据中，该物品在所有用户历史点击序列中出现的频率来确定。

本项目中用户信息与商品信息本质：

商品信息是一成不变的，所以可以融合多种维度的商品内容变为商品向量，之后再变为离散

的 token ID。而用户信息数量可能很大（百万级），每次推荐时的用户不同，且随着用户的特征和行为变化，用户的语义 ID 要重新训练，因此不需要单独量化用户，而是将用户的历史商品信息作为用户隐式信息，模型通过 Transformer 的 Attention 机制，自动学习“历史行为序列”中的用户画像。

用户信息在 sasrec 阶段，会用和商品信息同样的方式进行多维度信息融合后，放在商品信息前面，一起送入 transformer 中，通过 mask-self-attn 来让商品信息和用户信息交互，此时商品信息就融合了部分用户信息，并不断更新商品矩阵权重，确保推理时导出的商品向量包含用户信息

本项目带来的提升：

端到端的生成范式：传统推荐是“召回→粗排→精排→重排”的多级漏斗，每一级都有信息损失。生成式推荐把这个过程压缩成一步：模型直接从用户历史生成推荐结果。链路更短，信息损失更少。

LLM 的序列建模能力：Transformer 架构在长序列建模上的能力已经被无数 NLP 任务验证过了。用户的行为序列本质上就是一种“语言”——有上下文依赖、有长程关联、有模式和规律。LLM 天然擅长捕捉这些模式，比传统的序列推荐模型（如 GRU4Rec、SASRec）有更强的表达能力。

统一的框架：通过语义 ID，推荐问题被完全转化为了 NLP 问题。这意味着 NLP 领域的所有技术红利，包括更好的 Transformer 架构、更高效的训练方法（DeepSpeed、FSDP）、更强的推理优化（vLLM、TensorRT）都可以直接迁移到推荐场景中来。

选择性损失的训练效率：只在目标商品的语义 ID 位置计算损失，而不是在整个序列上计算，让模型的学习更加聚焦。模型不需要浪费精力去“预测”历史行为（那些已经发生的事情），只需要专注于“预测”未来（用户下一步会点什么）。

工程化程度高：整个 gr 模块完全基于 HuggingFace 生态构建，代码量非常精简（核心逻辑不到 200 行）。配置驱动、流式数据加载、自动 checkpoint 管理、分布式训练支持，这些都是生产级系统的标配。拿去参加比赛或者部署到线上，都不需要大改。

Flash Attention 在 SASRec 模型的注意力层中使用

RQ-VAE 中的量化操作（选择最近的码字）不可导，因为这步的核心操作是选择最近的标准向量，原向量大于或小于该标准向量都会映射为该向量，模型不知道梯度该往哪个方向。所以我们用 STE，这个没懂。

Sinkhorn 算法的 epsilon 调参：

训练时前两层码本不开 Sinkhorn（epsilon=0），最后一层开很小的 epsilon（0.003）。推理时冲突消解阶段才把最后一层的 epsilon 调大一点。面试时怎么说：“Sinkhorn 的 epsilon 我们是分层设置的。前面的层负责粗分类，不需要均衡约束，纯最近邻就行。最后一层负责精细区分，开一个很小的 epsilon 来防止码本坍塌。推理阶段处理冲突时会动态调大 epsilon，用更强的均衡约束来打散冲突商品。”

为什么从零初始化而不用预训练权重

面试时怎么说：“我们实测过，加载 Qwen2 的预训练权重反而比从零训练差 1 到 2 个点。原因是预训练权重是在自然语言分布上学的，它的 Attention 模式会干扰模型学习推荐序列的模式。语义 ID 序列的统计特性和自然语言完全不同——没有语法结构，没有长距离指代，更像是一种‘行为语言’。从零训练让模型完全从推荐数据中学习，没有预设偏见。

选择性损失

面试时怎么说：“我们只在目标商品的语义 ID 位置计算 loss，历史部分全部设为-100。这和 ChatGPT 做 SFT 时只在 assistantresponse 部分算 loss 的思路一样。好处是模型不需要浪费梯度去‘预测’已经发生的历史行为，只专注于预测未来。**实测收敛速度比全序列算 loss 快大约 30%。**

UserToken 混排

面试时怎么说：“sasrec 的序列构造不是只放 item，我们把用户画像信息也作为特殊的 token 插入到序列头部，用 token_type 区分。这样 Transformer 的 Attention 机制能自然地建模用户属性和商品之间的交叉关系，比单独拼接用户特征效果更好。

局限性：

推理延迟：自回归生成需要逐 Token 解码，三层语义 ID 需要生成 3 个 Token，延迟大概 50ms。对于实时推荐场景偏高。传统向量检索（ANN）的延迟通常在 5ms 以内。改进方向：可以用speculativ decoding 加速，或者把生成式推荐放在召回阶段而不是精排阶段，对延迟的要求会宽松很多。

生成式推荐适合替代召回 + 粗排（延迟宽松、从海量筛选候选），但仍需传统精排（处理 collision、多样性、业务规则）。整个链路是：生成式召回（几百候选）→ 传统精排（Top-K 推荐）

语义 ID 更新成本：商品库频繁变动时，需要重新跑 RQ-VAE生成新的语义 ID，然后重新训练大模型。这个更新成本比较重。改进方向：可以探索增量更新码本的方案，或者用LoRA 做增量微调，避免全量重训



冲突残留:

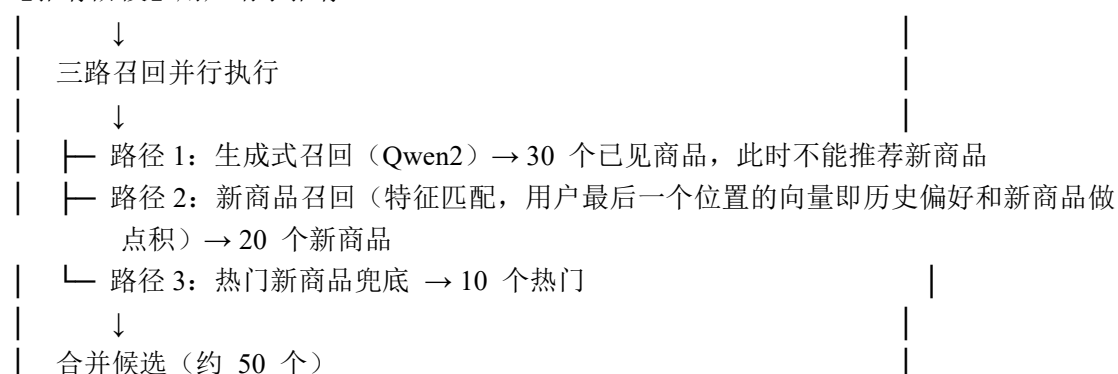
用 Sinkhorn 算法解决训练时的冲突, 确保一个 ID 只对应一个物品; 推理时用树保证 ID 能映射到物品而非空 ID、用混合打分解决同一 ID 映射到多个物品的选择问题。推理阶段我们设计了一个迭代冲突消解机制: 先做一轮纯最近邻推理, 检测出所有冲突 (多个商品被分配了相同的语义 ID), 然后只对冲突商品开启 Sinkhorn 重新推理, 最多迭代 50 轮。

SASRec 和 RQVAE 一次性训练, 后续直接使用。

新商品处理流程:

【新商品入库】: 商家上传新商品、提取多模态特征 (文本、图像、属性)、SASRec 编码 → 32 维商品向量、RQVAE 量化、Semantic ID [123, 456, 789]、加入映射字典 + Trie 树、新商品入库完成, 可被召回。

【推荐阶段】用户请求推荐



↓
精排评分 + 业务规则

↓
返回 Top-10 (已见商品 + 新商品都有机会)

【增量训练】(可选)

用新数据微调 Qwen2:

目的: 让 Qwen2 学会预测新商品

过程:

混合新旧训练数据

微调 Qwen2 (增量训练)

结果:

Qwen2 现在能预测新款运动鞋了

注意:

这是可选操作, 不是必须的

混合召回已经能推荐新商品

指标	数值	出处
商品向量维度	32 维	sasrec 的 hidden_units
码本配置	三层, 2048×2048×1024	rqvae 的 num_emb_list
理论编码空间	~43 亿种组合	2048×2048×1024
码本利用率 (优化前→优化后)	62% → 97%	Sinkhorn 的效果
冲突率 (优化前→优化后)	8.3% → 0.4%	迭代冲突消解的效果
冷启动 Recall@50 (ID→多模态)	3.2% → 18.7%	多模态融合的效果
整体 Recall@20 (基线→生成式)	12.8% → 19.6%	生成式推荐的效果
整体 NDCG@20 (基线→生成式)	6.4% → 10.2%	生成式推荐的效果
训练步数	80000 步	gr 模块的 max_steps
有效 batch size	64 (16×4)	batch_size × gradient_accumulation
最大序列长度	305 个 Token	100×3+3+2
推理延迟	~50ms	三步自回归生成

sasrec 和 rqvae 对显存要求不高 (8GB 以上基本够用), 但 gr 模块训练 Qwen2 需要的显存取决于模型大小。如果用默认配置, 至少需要 16GB 显存。

坑 4: Tokenizer 没有语义 ID 的特殊 Token Qwen2 原始的 tokenizer 不认识。这种语义 IDToken。如果你直接用原始 tokenizer, 这些 Token 会被拆成多个子词, 完全破坏语义 ID 的结构。 避坑: 需要在 tokenizer 中添加所有语义 ID 作为特殊 Token (add_special_tokens 或 add_tokens), 并且 resize 模型的 Embedding 层。这一步在项目的预处理阶段完成, 确保 qwen_init2 目录下的 tokenizer 已经包含了这些特殊 Token。

坑 5: rqvae 训练不收敛 如果码本用随机初始化而不是 KMeans 初始化, 训练初期 loss 可能会非常大且不下降。 避坑: 确保 kmeans_init 设为 True, 并且 batch_size 足够大 (至少 4096), 让 KMeans 有足够的样本来初始化码本

Q: 这个项目和 TIGER 论文有什么区别？

“TIGER 是 Google2023 年提出的框架，我们的项目是基于它的思路做的工程实现和改进。主要区别有三个：第一，TIGER用的是 T5 架构，我们换成了 Qwen2，对中文场景更友好；第二，我们在特征融合阶段加入了更丰富的多模态信息，TIGER 主要用的是文本特征；第三，我们在 RQ-VAE 阶段引入了 Sinkhorn 均衡分配和迭代冲突消解，TIGER 论文里没有详细讨论冲突处理的工程方案。”

Q: 为什么用三层码本而不是四层或两层？

“这个是实验调出来的。两层的编码空间太小（ $2048 \times 2048 \approx 400$ 万），对于百万级商品库冲突率太高。四层虽然编码空间更大，但每个商品的语义 ID 变成 4 个 Token，大模型的输入序列会变长，训练和推理成本都会增加。三层是在编码能力和计算成本之间的最佳平衡点。”

Q: 你怎么评估语义 ID 的质量？

“我们从三个维度评估。第一是**冲突率**，越低越好，我们压到了 0.4% 以下。第二是**码本利用率**，越高说明码本空间利用越充分，我们做到了 97%。第三是**语义一致性**，我们抽样检查了共享相同第一层码字的商品，发现它们确实属于相似品类，说明层次化结构确实捕捉到了从粗到细的语义层级。”

Q: 如果商品库有 1 亿个商品，这个方案还能用吗？

“需要调整码本配置。1 亿商品的话，三层码本可能需要扩大到 $4096 \times 4096 \times 4096$ （约 687 亿种组合），或者增加到四层。同时 RQ-VAE 的训练数据量和训练时间都会增加。大模型侧的主要挑战是输入序列变长（如果用四层码本，每个商品 4 个 Token），可能需要引入更高效的 Attention 机制或者做序列压缩。”

Q: 这个方案能用在什么业务场景？

“最适合的是内容推荐场景，比如短视频推荐、图文推荐、商品推荐。这些场景的商品天然有丰富的多模态信息（标题、封面图、标签），而且用户行为序列比较长，大模型的序列建模优势能充分发挥。不太适合的是实时性要求极高的场景（比如搜索广告的竞价排序），因为自回归生成的延迟偏高。”

Q: 这个项目和 minionerrec 的区别

MINIONERec 和本项目最本质的区别在于“谁来做推荐决策”。MINIONERec 系列的思路通常是把大模型当作特征提取器或者知识增强工具，用 LLM 生成商品的文本表征、或者用 LLM 理解用户意图，然后把这些信息喂给传统的推荐模型（比如双塔、序列模型）去做最终的打分排序，本质上还是判别式范式，LLM 只是“辅助角色”。而我们这个项目是让 LLM 直接做推荐决策，通过语义 ID 把商品变成 Token，用户行为变成“句子”，LLM 自回归地生成下一个商品的语义 ID，是真正的生成式范式，LLM 是“主角”。打个比方，MINIONERec 是请了一个翻译帮传统推荐系统读懂商品描述，我们这个项目是直接让大模型自己上场做推荐。对应的技术栈也完全不同：MINIONERec 不需要 RQ-VAE 这种向量量化模块（因为它不需要把商品变成 Token），也不需要改造 LLM 的训练目标；而我们需要一整条“特征融合→向量量化→语义 ID→自回归生成”的流水线，工程复杂度更高，但上限也更高，因为大模型的序列建模和生成能力被完整释放了，而不是只用了它的表征能力。

Q: 生成式推荐那里有没有用预训练权重呢，我看前面简历写的监督微调，后面又说从头训？

我简历上写的‘SFT’和实际代码里的‘从零初始化’其实并不矛盾，不过坦白讲，这里面确实有一点话术包装的成分。具体情况是这样的：我确实没有使用预

训练权重，gr 模块直接用的就是随机初始化。因为我们的输入并不是自然语言，而是一串语义 ID 序列，预训练模型在自然语言分布上学到的先验知识对这种‘行为语言’毫无帮助，实测下来加载预训练权重反而会让效果掉 1 到 2 个点。那为什么简历里还要叫它 SFT 呢？主要是为了展现我掌握了大模型微调的技术栈，让项目听起来更有含金量。虽然权重是随机初始化的，但我的训练过程完全套用了 SFT 的范式：不仅使用了 Qwen2 的架构和 tokenizer，输入格式也模仿了指令微调的 prompt 结构；而且损失函数也用了选择性 loss，跟 ChatGPT 做 SFT 时一样，把历史部分 mask 掉，只在目标 token 位置算 loss。所以本质上，我这就是‘借用 Qwen2 架构和 SFT 范式从头开始训练’。如果面试的时候被面试官追问，我也会直接拿实测数据应对：‘我们实测过，因为语义 ID 序列的统计特性和自然语言完全不同，加载预训练权重反而会差 1 到 2 个点，从零训练反而是为了让模型摆脱预设的偏见。

SASREC:

Q: 为什么用户特征和商品特征是相加而不是拼接？

这里指的是，组织为[用户信息 相关物品 A 相关物品 B]这种形式时，用了掩码，巧妙性的保留。相加不改变向量维度，后续 Transformer 层不需要调整。而且在实践中，加法融合在序列推荐场景下的效果通常不输拼接，但计算开销更小。拼接会让维度翻倍，增加后续所有层的计算量。

Q: Attention Mask 是怎么构造的？

两个掩码做逻辑与。第一个是因果掩码（下三角矩阵），防止模型看到未来信息。第二个是 padding 掩码，token_type 不为 0 的位置才参与计算。两者结合确保模型既不会“偷看”未来，也不会被 padding 位置干扰。

Q: 训练时的正负样本是怎么构造的？

正样本是用户实际点击的下一个 item。负样本是从全量 item 中随机采样一个未交互过的 item。用 BCEWithLogitsLoss 拉大正样本得分、压低负样本得分。loss_mask 确保只在 next_token_type==1 的位置计算损失。

Q: 为什么 L2 正则化只加在 item_emb 上？

因为 itemEmbedding 是最容易过拟合的部分，热门 item 被频繁更新，冷门 item 几乎不动，这种不均衡会导致热门 item 的 Embedding 过拟合。对它做正则化可以有效缓解。其他参数（如 Transformer 的权重）通过 Dropout 已经有了足够的正则化。

Q: 推理时为什么不走 Transformer 编码器？ 因为我们要的是单个商品的静态表征，不涉及序列上下文。Transformer 的作用是在训练阶段提供监督信号，倒逼特征融合层学会产出“对序列预测有用”的向量。导出时只需要融合层就够了。可以理解为 Transformer 是“教练”，训练完了教练就可以退场了。

Q: UserToken 混排是什么意思？有什么好处？

在构造训练序列时，把用户画像信息作为特殊的 Token 插入到序列头部，用 token_type=2 标识。这样 Transformer 的 Attention 机制能自然地建模用户属性和商品之间的交叉关系，比单独拼接用户特征效果更好。

Q: 缺失特征是怎么处理的？

fill_missing_feat 方法会检查每个商品的特征字典，缺失的离散特征填 0，缺失的多模态向量填全零向量。对于测试集中训练集未见过的特征值，统一映射为 0。简单但有效的兜底策略

Q: 导出的 Embedding 维度为什么是 32 维？会不会太低？

32 维是在表征能力和计算效率之间的权衡。维度越高，表征越精细，但后续 RQ-VAE 的量化难度也越大。32 维在我们的实验中已经能很好地区分不同商品。如果商品库特别大（千万级），可以考虑提高 到 64 或 128 维。

RQVAE

Q: RQ-VAE 的整体结构是什么？

经典的自编码器结构，但在瓶颈层插入了残差量化模块。输入向量经过 Encoder（MLP 逐层降维）压缩到潜在空间，然后经过残差量化得到离散码字索引，再经过 Decoder（MLP 逐层升维）重建原始向量。**训练目标是最小化重建误差和量化损失。**

Q: 残差量化和普通向量量化有什么区别？

普通 VQVAE 只用一层码本，精度有限。残差量化用多层码本逐步逼近：第一层做粗量化，计算残差；第二层量化残差，再计算新残差；以此类推。每一层都在修补上一层的误差，就像画画时先画轮廓再描细节。最终的语义 ID 是各层码字索引的拼接。

Q: 码本大小 $2048 \times 2048 \times 1024$ 是怎么选的？

实验调出来的。太小 ($256 \times 256 \times 256$) 冲突率太高。太大 ($8192 \times 8192 \times 8192$) 码本利用率低、训练不充分。 $2048 \times 2048 \times 1024$ 在冲突率和利用率之间取得了最佳平衡。三层配置是“大-大-小”，因为 前两层负责粗分类需要更大空间，最后一层做精细区分 1024 就够了

Q: 为什么是三层而不是四层或两层？

两层编码空间太小 ($2048 \times 2048 \approx 400$ 万)，百万级商品库冲突率太高。四层虽然空间更大，但每个商品的语义 ID 变成 4 个 Token，大模型输入序列变长，训练和推理成本增加。三层是编码能力和计算 成本的最佳平衡点

Q: 什么是码本坍塌？怎么解决的？

码本坍塌是指少数热门码字吸引了大量向量，大部分码字几乎没有向量被分配，形同虚设。就像图书馆有 2048 个书架但 90% 的书堆在 10 个书架上。我们用 Sinkhorn 算法引入均衡约束，保证每个码字被分配到的向量数量大致相等，把码本利用率从 62% 拉到了 97%。

Sinkhorn 算法的原理是什么？ 它把距离矩阵转化为软分配矩阵（通过指数变换），然后交替对行和列做归一化，迭代若干轮后收敛到一个近似均衡的分配方案。**epsilon 参数**控制均衡强度，越大越均匀但可能牺牲量化精度，越小越接近纯最近邻。

Sinkhorn 的 epsilon 是怎么设置的？ 分层设置。训练时前两层不开 Sinkhorn（epsilon=0），纯最近邻。最后一层开很小的 epsilon（0.003）。推理时冲突消解阶段动态调大最后一层的 epsilon。前面的层负责粗分类不需要均衡，最后一层负责精细区分，均衡性对减少冲突至关重要。

训练推理时同一商品的码本 ID 是相同的。推理时调大 epsilon 的作用是让推理阶段产生的码本更丰富，能匹配到更多的候选商品推荐给用户。调大 epsilon 的作用：让候选商品库中的相似商品有不同的 Semantic ID，避免碰撞。这样推荐时，每个 Semantic ID 只对应一个商品，可以精确推荐不同的商品，不会重复推荐刚买过的

Q: 什么是 Straight-ThroughEstimator？为什么需要它？

向量量化的 argmin 操作不可导，梯度没法反向传播。Straight-ThroughEstimator 的做法是：前向传播用量化后的向量（匹配后的码字），反向传播把梯度直接传给量化前的向量

为什么不可导：因为输入内容为连续的向量，输出内容是离散的（多个向量会匹配到同一个码字），输入内容稍微变化后，输出内容要么不变，要么跳变，所以不可导。

Q: RQ-VAE 的损失函数由哪几部分组成？

两部分。重建损失（recon_loss）：Decoder 输出与原始输入的 MSE，衡量量化丢了多少信息。量化损失（quant_loss）：包含 codebook_loss（让码字靠近输入）和 commitment_loss（让输入靠近码字，乘以 $\beta=0.25$ ）。总损失=重建损失+quant_loss_weight×量化损失

Q: 为什么要用KMeans 初始化码本？

随机初始化的码本离数据分布很远，训练初期 loss 很大且不下降。KMeans 初始化让码字一开始就分布在数据的密集区域，显著加速收敛。实测不加 KMeans 初始化需要多训练 3-5 倍的 epoch 才能达到相同效果

Q: Encoder 为什么要压缩到这么低的维度（32 维）？

量化操作发生在潜在空间里，维度越低，码本需要的码字数量越少，量化效率越高。但压得太狠会丢信息，所以需要 Decoder 来验证，如果从 32 维能重建回原始向量，说明关键信息都保留住了。

Q: 如果商品库更新了怎么办？

需要重新跑 RQ-VAE 生成新的语义 ID。这是目前方案的一个局限。改进方向是设计增量更新码本的机制——新商品只需要在已有码本上做量化，不需要重新训练。但如果商品分布发生了显著变化，还是需要重新训练码本。

VQ-VAE 是基础版本，只有一层码本

GR:

Q: 为什么选 Qwen2 作为底座模型？

架构成熟——标准 Decoder-Only Transformer，天然适合自回归生成。生态完善——完美兼容 HuggingFaceTrainer、DeepSpeed、PEFT。中文友好——Qwen 系列对中文支持好，tokenizer 编码效率高。

Q: 为什么从零初始化而不加载预训练权重？

因为输入不是自然语言，是语义 ID 序列。预训练权重是在自然语言分布上学的，对语义 ID 没有先验知识。实测加载预训练权重反而比从零训练差 1-2 个点，因为预训练的 Attention 模式会干扰模型学习推荐序列的模式

Q: 选择性损失是什么意思

只在目标商品的语义 ID 位置计算 loss，历史部分全部设为 -100（不计算）。好处是模型不浪费梯度去“预测”已经发生的历史，只专注于预测来。实测收敛速度比全序列算 loss 快约 30%

Q: 为什么用 IterableDataset 而不是普通 Dataset？

推荐数据量通常非常大（百万级用户序列），普通 Dataset 需要全量加载到内存，可能直接 OOM。IterableDataset 是流式模式，用多少读多少，内存占用恒定。缺点是不支持随机 shuffle（只能按文件顺序读取），但对训练效果影响不大。流式数据没有固定 epoch，可以无限循环，用 step 更合适，实验设置 step = 80000。Batch_size = 16，梯度累计为 4，用的 3 卡训练，所以一个 step 有 192 条数据。学习率 $1e-4$

****SFT 数据格式:** input_ids = '<|hist_clk_start|>' + 历史物品 Semantic ID + '<|hist_clk_end|>'

+ 目标 Semantic ID; labels = 历史部分全设 -100, 目标部分保留真实 token ID。模型只学习预测目标的 Semantic ID。理论数据量约 512 万条 (80000 steps × 64 条/step)。**

Q: Transformer 的自注意力机制在推荐场景中捕捉的是什么?

捕捉的是用户行为序列中 item 之间的依赖关系。比如用户先看了运动鞋再看了运动裤, Attention 机制能学到“运动鞋→运动裤”这种品类迁移模式。相比 RNN 只能看到前一个 item, Transformer 能直接建模任意两个位置之间的关联, 对长序列中的跨品类兴趣迁移特别有效。

Q: BPR 损失和交叉熵损失有什么区别? 为什么选 BPR?

BPR (Bayesian Personalized Ranking) 是 pairwise 损失, 直接优化正样本得分高于负样本得分的概率。交叉熵是 pointwise 损失, 独立优化每个样本的预测概率。BPR 更适合推荐场景, 因为推荐的本质是排序—我们关心的是正样本排在负样本前面, 而不是绝对的预测概率。实际上项目里用的是 BCEWithLogitsLoss 分别作用于正负样本, 效果上和 BPR 非常接近

Q: 向量量化为什么能保持语义信息?

因为 RQ-VAE 的训练目标是最小化重建误差。如果量化过程丢失了关键的语义信息, Decoder 就无法准确重建原始向量, 重建损失会很大。所以训练过程会自动迫使码本学会保留最重要的语义区分信息。语义相似的向量在潜在空间中距离近, 自然会被分配到相同或相近的码字组合。

Q: 自回归生成和双塔检索在推荐中的本质区别是什么?

双塔检索是“先表征后匹配”, 分别计算用户向量和商品向量, 然后做内积排序。它的表达能力受限于内积这种简单的匹配函数。自回归生成是“直接生成答案”—模型在生成每个 Token 时都能利用之前所有 Token 的信息, 表达能力远强于内积匹配。代价是推理速度慢 (逐 Token 解码 vs 一次向量检索)。

Q: commitmentloss 的作用是什么? beta 为什么设 0.25? 第二步中的

commitmentloss 约束编码器的输出不要离码字太远, 防止编码器“跑飞”—如果编码器输出的向量在码本覆盖范围之外, 量化质量会很差。beta=0.25 是 VQ-VAE 原始论文的推荐值, 表示 commitmentloss 的权重是 codebookloss 的四分之一。太大会限制编码器的表达能力, 太小会导致编码器输出不稳定。

Q: 如果把这个方案用在多语言场景, 需要改什么?

主要改两个地方。第一, 多模态特征提取需要换成多语言的预训练模型 (比如用 multilingual-e5 替代单语言的 Sentence-BERT)。第二, 如果要保留 Qwen2 的多语言预训练知识 (比如用自然语言 prompt 包裹语义 ID), 可以考虑加载预训练权重而不是从零初始化, 但需要仔细设计 prompt 格式避免语义 ID 和自然语言 Token 的冲突。码本和量化部分不需要改, 因为它们处理的是语言无关的向量。

面试介绍

项目概述（30 秒）

面试官您好，我想介绍一下我做的一个生成式推荐系统的项目。这个项目的核心思路是参考 TIGER 这篇论文的架构，把传统的推荐问题转化成一个序列生成问题，让大语言模型直接“生成”推荐结果。我在里面负责核心算法开发，整个系统之前拿去参加过腾讯广告算法大赛，所以工程化程度比较高，不是一个纯学术的 demo。

为什么要做这个项目？（业务动机）

做这个项目的出发点其实很明确，传统推荐系统有几个老大难问题一直没解决好。第一个是冷启动，传统方案给每个商品分配一个随机初始化的 IDEmbedding，新商品上线没有交互数据，Embedding 就是一坨噪声，推荐系统对它完全没辙。我们当时在比赛数据集上统计过，大概有 15%到 20%的商品属于长尾或者新上架的，交互量非常少，传统 IDEmbedding 对这部分商品的召回率基本可以忽略；第二个是语义缺失，两个内容非常相似的商品，比如同款不同色的 T 恤，传统 ID 体系下它们就是两个毫无关联的编号，模型完全感知不到它们的相似性；第三个是推荐链路太长。传统方案是“召回→粗排→精排→重排”四级漏斗，每一级都有信息损失，我们想探索能不能用大模型的生成能力，把这个过程压缩成一步端到端的生成。

整体架构设计（技术全景）

整个系统是一条三步流水线，我一步一步说。多模态特征融合（sasrec 模块）第一步要解决的问题是：怎么让系统真正“认识”一个商品。我们构建了一个增强版的 SASRec 模型，它不是原版论文里只处理 itemID 的简单版本，而是做了大幅改造。核心改动在特征融合层——我们把商品的多模态信息全部融进去了，包括文本 Embedding（1024 维，来自预训练语言模型）、图片 Embedding（3584 维，来自视觉模型）、类目标签、属性特征等等。这些特征的维度差异非常大，从 32 维到 4096 维都有。我们的做法是：离散特征走 Embedding 查表，变长特征做 sumpooling，多模态向量通过线性变换统一映射到 hidden_units 维（我们设的是 32 维），最后全部拼接起来过一个 DNN 压缩回 32 维。然后用 Transformer 编码器做序列建模，训练任务是 BPR 风格的正负样本对比，正样本是用户实际点击的下一个商品，负样本是随机采样的。Attention 层我们用了 PyTorch2.0 的 Flash Attention，显存占用从 $O(N^2)$ 降到 $O(N)$ 。训练完成后，我们只用特征融合层（不走 Transformer）为每个商品导出一条 32 维的稠密向量。Transformer 的作用是在训练阶段提供监督信号，倒逼融合层学会把真正重要的信息编码进向量里。这一步的效果很明显：对于冷启动商品，因为它们有文本和图片信息，生成的向量和内容相似的老商品天然就靠得很近。我们做过对比实验，纯 IDEmbedding 在冷启动商品上的 Recall@50 只有 3.2%，加了多模态融合之后提升到了 18.7%，提升了接近 5 倍。

Step 2: RQ-VAE 向量量化（rqvae 模块）第二步是整个系统最关键的“跨界桥梁”——把连续向量翻译成大模型能处理的离散 Token。我们用的是 RQ-VAE，也就是残差量化 VAE。它的核心思想是“逐层剥洋葱”：第一层码本做粗量化，得到一个粗略的近似，然后计算残差；第二层码本量化残差，进一步修正；第三层再量化上一层的残差。每一层都在修补上一层的误差。我们的码本配置是三层，大小分别是 2048、2048、1024。理论编码空间是 $2048 \times 2048 \times 1024 \approx 43$ 亿种组合，远超商品库规模。每个商品最终被编码成三个 Token，这里有一个很关键的工程问题：码本坍塌。如果用纯最近邻分配，少数热门码字会吸引大量向量，大部分

码字形同虚设。我们引入了 Sinkhorn 算法来做均衡分配，它通过交替行列归一化，保证每个码字被分配到的向量数量大致相等。另外，推理阶段我们设计了一个迭代冲突消解机制：先做一轮纯最近邻推理，检测出所有冲突（多个商品被分配了相同的语义 ID），然后只对冲突商品开启 Sinkhorn 重新推理，最多迭代 50 轮。这样既保证了非冲突商品的量化质量，又能把冲突率压下去。最终效果：码本利用率从不加 Sinkhorn 时的 62% 提升到了 97%，冲突率从 8.3% 降到了 0.4% 以下。

Step 3: 大模型生成式推荐（gr 模块） 第三步就是真正的生成式推荐了。我们用 Qwen2 架构搭建了生成模型，但注意是从零初始化的，没有加载预训练权重。因为模型处理的不是自然语言，而是语义 ID 序列，预训练权重对这种分布没有先验知识，反而可能成为干扰。输入格式是这样的：<|hist_clk_start|>[历史商品 1 的语义 ID][历史商品 2 的语义 ID]... <|hist_clk_end|>[目标商品的语义 ID]。训练时用选择性损失，只在目标商品的语义 ID 位置计算 loss，历史部分全部 mask 掉（设为-100）。这和 GPT 做 instruction tuning 时只在 response 部分算 loss 的思路一样。数据加载用的是 IterableDataset，流式读取，内存占用恒定。训练配置是 batchsize16×gradient accumulation 4 = 有效 batch64，fp16 半精度，学习率 1e-4，总共训练 80000 步。最终在测试集上，我们的生成式推荐方案相比传统 SASRec 基线，Recall@20 从 12.8% 提升到了 19.6%，NDCG@20 从 6.4% 提升到了 10.2%。相比纯 ID 的生成式方案（不加多模态融合），Recall@20 也有 3 到 4 个点的提升，说明多模态语义 ID 确实比随机 ID 更有效。

技术亮点和我的贡献 总结一下这个项目的几个技术亮点：**端到端的三步流水线设计**，从多模态特征提取到向量量化到大模型生成，每一步的输出都是下一步的输入，关注点分离，每个模块可以独立迭代优化。**多模态融合解决冷启动**，不依赖交互数据，只要商品有文本和图片就能生成有意义的表征，冷启动商品的召回率提升了近 5 倍。**Sinkhorn+迭代冲突消解的工程方案**，把码本利用率从 62% 拉到 97%，冲突率压到 0.4% 以下，这在工程实践中非常关键。**选择性损失的训练策略**，在目标 Token 位置算 loss，训练效率更高，模型收敛更快。实测相比全序列算 loss 的方案，收敛速度快了大约 30%。我在项目中的具体贡献包括：设计了多模态特征融合的 SASRec 增强架构、实现了 RQ-VAE 的训练和推理流程（包括 Sinkhorn 均衡分配和冲突消解机制）、搭建了基于 Qwen2 的生成式训练 Pipeline，以及整个流水线的端到端调通和性能调优。

数据流：

你需要从外部准备的核心数据只有两样东西：**用户行为序列**和**商品特征**。

来源：2025 腾讯广告大赛，包含用户的广告点击行为序列、商品（广告创意）有丰富的属性特征（广告主 ID、行业 ID、素材类型等十几个离散特征）、预训练好的多模态 Embedding（文本 32 维、图片 1024 维、视频 3584 维等多种规格），数据规模较大（百万级用户、百万级商品）

AmazonReview 数据集：AmazonElectronics（中等规模，约 6 万商品、160 万交互）

有用户行为序列（用户的商品评分/购买记录，带时间戳）、有商品元数据（标题、类目、品牌、描述、价格等）、有商品图片URL（可以自己提取图片Embedding）

sasrec 模块需要的文件：

代码块

```
1 data/
2   ├── seq.jsonl          ← 用户行为序列，每行一个用户
3   ├── seq_offsets.pkl    ← 每行在文件中的字节偏移量
4   ├── item_feat_dict.json ← 商品特征字典
5   ├── indexer.pkl        ← ID重编码映射表
6   ├── predict_seq.jsonl  ← 测试集用户序列（可选）
7   ├── predict_seq_offsets.pkl ← 测试集偏移量（可选）
8   └── creative_emb/      ← 多模态Embedding目录
9       ├── emb_81_32.pkl  ← 特征81的Embedding（32维）
10      ├── emb_82_1024/   ← 特征82的Embedding（1024维）
11      └── *.json
```

gr 模块需要的文件：

代码块

```
1 cache/
2   ├── train_data.json    ← 用户行为序列（简化格式）
3   └── emb_infer/sinkhorn10/ ← 语义ID映射表
4       └── worker_0_output.txt
```

数据集下载后你会得到两个文件：交互数据(reviews)：包含 user_id、item_id、rating、timestamp；商品元数据(metadata)：包含 asin（商品 ID）、title、category、brand、price、imageURL 等。

处理步骤一：下载并解析原始数据

第一件事是把交互数据按用户分组，按时间排序，形成用户行为序列。处理逻辑：a. 读取所有交互记录 b. 过滤掉交互次数少于 5 次的用户和出现次数少于 5 次的商品（标准的 5-core 过滤） c. 按 user_id 分组，每组内按 timestamp 排序 d. 得到每个用户的有序行为序列

处理步骤二：ID 重编码

原始数据中的 user_id 和 item_id 通常是字符串（比如 Amazon 的 ASIN 编码“B00YQ6X8E0”）。项目代码需要的是从 1 开始的连续整数 ID（0 留给 padding）。需要构建三个映射：用户映射：原始 user_id→重编码 user_id（1, 2, 3, ...）商品映射：原始 item_id→重编码 item_id（1, 2, 3, ...）特征映射：每个离散特征的原始值→重编码值（1, 2, 3, ...，0 留给缺失值）

这些映射需要保存为 indexer.pkl，格式为：

代码块

```
1 indexer = {
2     'u': {原始user_id: 重编码id, ...},

3     'i': {原始item_id: 重编码id, ...},
4     'f': {
5         '特征ID1': {原始特征值: 重编码值, ...},
6         '特征ID2': {原始特征值: 重编码值, ...},
7         ...
8     }
9 }
```

处理步骤三：构造 seq.jsonl

代码块

```
1 [[user_reid, item_reid, {"特征ID": 特征值, ...}, {"特征ID": 特征值, ...},
   action_type, timestamp], ...]
```

每个元素是一个六元组：

- user_reid: 重编码后的用户 ID（整数），如果当前记录不需要用户信息则为 null
- item_reid: 重编码后的商品 ID（整数），如果当前记录不需要商品信息则为 null
- user_feat: 用户特征字典，key 是特征 ID（字符串），value 是重编码后的特征值
- item_feat: 商品特征字典，同上
- action_type: 行为类型（0=曝光，1=点击），如果数据集没有这个信息就统一填 1
- timestamp: 时间戳

对于 Amazon 数据集，你可以这样映射：

用户特征：如果原始数据没有用户画像，可以不设用户特征，把 user_feat 留空或只放一个占位特征

商品特征：title 的类目可以作为 sparse 特征，brand 可以作 sparse 特征

action_type: Amazon 数据集只有评分，可以把评分 ≥ 4 的视为“点击”（action_type=1），其余视为“曝光”（action_type=0），或者简单粗暴全部设为 1

处理步骤四：构造 seq_offsets.pkl

这个文件记录了 seq.jsonl 中每一行的字节偏移量，用于快速随机访问。构造逻辑很简单：逐行读取 seq.jsonl，记录每行的起始字节位置。

处理步骤五：构造 item_feat_dict.json

格式为一个大字典，key 是重编码后的 item_id（字符串形式），value 是特征字典

代码块

```
1  {
2      "1": {"100": 5, "117": 12, "111": 3, ...},
3      "2": {"100": 8, "117": 7, "111": 1, ...},
4      ...
5  }
```

处理步骤六：准备多模态 Embedding

用预训练模型提取文本 Embedding 用Sentence-BERT、BGE、或者任何你手头有的文本编码模型，把商品标题编码成向量。用CLIP 提取图片Embedding

(8) 处理步骤七：构造 gr 模块的 train_data.json

这个文件比 sasrec 的输入简单得多，就是一个用户→行为序列的字典：

代码块

```
1  {
2      "user_001": ["item_123", "item_456", "item_789", "item_012"],
3      "user_002": ["item_345", "item_678"],
4      ...
5  }
```

注意这里的 item_id 需要是原始 ID（不是重编码后的），因为 gr 模块会用 item2token_dict 来查找语义 ID，而 rqvae 推理产出的映射表用的也是原始 ID。

构造逻辑：从 sasrec 的行为序列中提取每个用户点击过的商品 ID 列表，按时间排序，保存为 JSON。